

Informatik 2

Kapitel 4 – Java-Grundlagen und Objektorientierung

Inhaltsverzeichnis

4.1	Einleitung	2
4.2	Klassen und Objekte	10
4.2.1	Klassen in Java	11
4.2.2	Attributliste	12
4.2.3	Konstruktorliste	15
4.2.4	Methodenliste	18
4.2.5	Objekte in Java	21
4.2.6	Vergleich von Java und C	28
4.3	Vererbung	28
4.3.1	Die Klasse <code>Object</code>	33
4.3.2	Oberklassen	34
4.3.3	Substitutionsprinzip	35
4.3.4	Polymorphismus	38
4.3.5	Typ- und Klassenzugehörigkeit	40
4.3.6	Abstrakte Klassen	41
4.3.7	Fallstudie: Grafischer Editor	43
4.3.8	Die <code>super</code> -Referenz	46
4.4	Schnittstellen (Interfaces)	47
4.4.1	Deklaration und Definition von Schnittstellen	50
4.4.2	Schnittstellen als Datentypen	52
4.5	Arrays	54
4.5.1	Mehrdimensionale Arrays	57
4.5.2	Die Klasse <code>Arrays</code>	59
4.5.3	Vergleich von Java und C	59

4.1 Einleitung

In diesem Kapitel wollen wir systematisch in die Grundlagen und wesentlichen Prinzipien der objektorientierten Programmierung anhand der Programmiersprache Java einführen. Nach den ersten Beispielen, die wir dazu bereits in Kapitel 2 kennengelernt haben, werden wir hier weitere Beispiele aus der Java-API als auch weitere eigene Klassen betrachten.

Einleitend wollen wir exemplarisch eine eigene Klasse `Person` für eine Partnervermittlungsanwendung entwickeln. Wir beginnen mit einer Darstellung der Klasse mit ihren Attributen als UML-Klassenkarte in Abbildung 1.

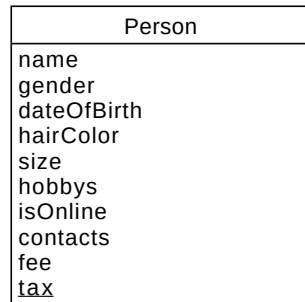


Abbildung 1: Klassenkarte Person mit Attributen.

Ein Objekt der Klasse hat für jedes Attribut zu einem bestimmten Zeitpunkt einen Datenwert. Diese Datenwerte können sich im Lauf der Zeit ändern (wie `hairColor`), oder auch nicht (wie `dateOfBirth`). Für Klassenattribute ist der Datenwert für alle Objekte der Klasse immer gleich (wie `tax`), für Objektattribute kann der Datenwert für verschiedene Objekte der Klasse unterschiedlich sein (alle anderen). Ein `Person`-Objekt könnte in Form einer Objektkarte wie in Abbildung 2 aussehen.

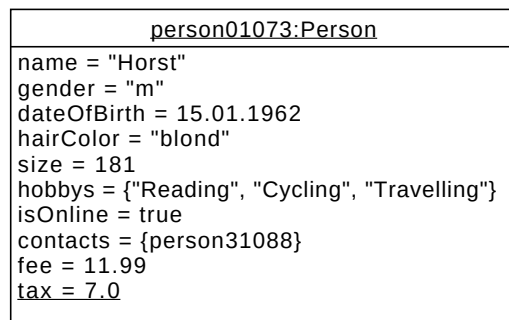


Abbildung 2: Objektkarte zur Klasse Person.

In Java lässt sich die Klassenkarte `Person` wie folgt implementieren:

```

package Kapitel4;

import java.time.LocalDate;

public class Person {

    private String name;
    private String gender;
    private LocalDate dateOfBirth;
    private String hairColor;
    private int size;
    private String[] hobbies = new String[5];
    private boolean isOnline;
    private Person[] contacts = new Person[10];
    private double fee;
    private static double tax = 7.0;
}

```

Vor der Klassendefinition legen wir als Erstes ein Paket an, in das unsere Klasse kommt. Mit Paketen können wir unser Programm besser strukturieren. Als nächstes importieren wir alle API-Klassen, die wir benötigen, und die sich nicht im Paket `java.lang` (dieses ist immer automatisch eingebunden) befinden. Dann kommt die Definition der Klasse. Im Rumpf der Klasse wird jedes Attribut wie eine Variable deklariert. Für die Deklaration müssen wir uns für jedes Attribut einen passenden Datentyp überlegen. Hier dürfen wir nicht nur primitive Datentypen wie `int` verwenden, sondern auch beliebige passende Klassen aus der Java-API. Für einige Objektarten der realen Welt bietet die Java-API dazu bereits anwendungsunabhängige Vorlagen an, wie hier die Klassen `LocalDate` für Datum-Objekte und `String` für String-Objekte. Da die Attribute `hobbies` und `contacts` jeweils mehrere Werte haben können, legen wir diese als Arrays an. Solche Attribute nennt man **mehrwertig**. Wie man am Attribut `contacts` zudem sieht, können wir auch unsere neue Klasse `Person` gleich als Datentyp verwenden. Klassenattribute werden mit `static` angegeben.

Wir können direkt in der Deklaration eine Wertzuweisung vornehmen. Das machen wir, wenn es für ein Attribut einen einheitlichen anfänglichen Standardwert gibt. Das trifft insbesondere auf die beiden mehrwertigen Attribute `hobbies` und `contacts` zu, die zu Beginn als leere Arrays initialisiert werden, in die noch keine Werte eingetragen sind. Dies macht man wie bei einem Konstruktorauf-ruf. Hierbei müssen wir in den eckigen Klammern die Anzahl der Elemente der Arrays festlegen. Diese entspricht dann der maximalen Anzahl an Werten, die man für das jeweilige Attribut speichern kann (das sind im Beispiel maximal 5 Hobbys und 10 Kontakte).

Nach dem Prinzip der Datenkapselung nach David Parnas (Kapitel 3) gehören die Attribute grundsätzlich zur internen Implementierung einer Klasse, so dass wir deren Sichtbarkeit standardmäßig als `private` festlegen (und die wir so vor dem direkten Zugriff von außen schützen). Den Zugriff erlauben wir nur indirekt über die Methoden einer Klasse (diese diskutieren wir als nächstes). In der Implementierung der Methoden können wir sicherstellen, dass Attribute nur gültige Werte erhalten und keine Abhängigkeiten zwischen den Werten verschiedener Attribute verletzt werden. Solche Eigenschaften von Attributwerten bezeichnen wir als **Datenstrukturinvarianten**. Für die Klasse `Person` wären solche Datenstrukturinvarianten zum Beispiel (überlegen Sie weitere mögliche Datenstrukturinvarianten):

- `size` muss eine positive Zahl sein.
- `name` muss mit einem Großbuchstaben beginnen.

Um auf die Daten eines Objekts zuzugreifen und diese verwalten zu können, benötigen wir als nächstes Methoden. Diese sind der Klasse zugeordnet. Die Attribute bilden zusammen mit den Methoden eine Programmeinheit, die wir in Kapitel 3 Modul genannt haben. Wir ergänzen dazu in Abbildung 3 zuerst die Klassenkarte um sinnvolle Methoden für Personen in einer Partnervermittlungsanwendung. Mit Get-Methoden können wir die in den jeweiligen Attributen gespeicherten Datenwerte lesen. Mit den Set-Methoden können wir die in den jeweiligen Attributen gespeicherten Datenwerte überschreiben. Das sind Standardmethoden, die man immer braucht. Get- und Set-Methoden sind Objektmethoden, wenn sie zu Objektattributen gehören, und Klassenmethoden, wenn sie zu Klassenattributen gehören.

Ist ein Attribut mehrwertig (zum Beispiel `hobbys`), weicht man mit der Namensgebung der zugehörigen Zugriffsmethoden oft etwas ab, um die Mehrwertigkeit im Namen deutlich zu machen. Mit Add-Methoden wird in diesem Fall ein Wert hinzugefügt, mit Remove-Methoden ein Werte entfernt (zum Beispiel `addHobbys`, `removeHobbys`). Schließlich sind mit `login` und `logout` noch Methoden aufgelistet, die Benutzeraktionen in der Anwendung beschreiben.

Person
name gender dateOfBirth hairColor size hobbys isOnline contacts fee tax
setName getName setGender getGender setDateOfBirth getDateOfBirth setHairColor getHairColor setSize getSize addHobbys removeHobbys login logout addContact removeContact getFee setFee getTax setTax

Abbildung 3: Klassenkarte Person mit Methoden.

Man sieht, dass hier sehr viele Methoden zusammenkommen. Die Java-Implementierung aller Methoden im Skript zu zeigen, wäre unübersichtlich. Zudem sind alle Getter und alle Setter analog implementiert. Für eine erste exemplarische (und noch vereinfachte) Darstellung in Java mit Methoden wählen wir deshalb nur drei einwertige Attribute und ein mehrwertiges Attribut aus:

```
package Kapitel4;

import java.time.LocalDate;

public class Person {

    private LocalDate dateOfBirth;
    private int size;
    private String[] hobbies = new String[5];
    private static double tax = 7.0;

    public Person(LocalDate dateOfBirth, int size) {
        setGeburtsdatum(dateOfBirth);
        setGroesse(size);
    }

    public LocalDate getDateOfBirth() {
        return dateOfBirth;
    }

    private void setDateOfBirth(LocalDate dateOfBirth) {
        this.dateOfBirth = dateOfBirth;
    }

    public int getSize() {
        return size;
    }

    public void setSize(int size) {
        this.size = size;
    }

    public void addHobbys(String hobby, int position) {
        if (position < this.hobbys.length)
            this.hobbys[position] = hobby;
    }

    public void removeHobbys(int position) {
        if (position < this.hobbys.length)
            this.hobbys[position] = null;
    }

    public static double getTax() {
        return tax;
    }

    public static void setTax(double tax) {
```

```

        Person.tax = tax;
    }
}

```

Klassenmethoden werden mit `static` angegeben. Es fällt auf, dass wir für `setDateOfBirth` die Sichtbarkeit `private` gewählt haben. Damit ist `setDateOfBirth` in anderen Klassen nicht aufrufbar. Das ist Absicht, denn das Geburtsdatum soll nur einmal zu Beginn intern gesetzt werden dürfen, und darf dann nicht mehr von außen änderbar sein.

Die dargestellte Implementierung ist noch in der Hinsicht vereinfacht, dass wir bisher in den Settern keine Datenstrukturinvarianten berücksichtigen. Zum Beispiel akzeptieren wir auch, dass an `setSize` ein negativer Wert übergeben wird. Um dieses Problem zu lösen, könnten wir vor der Wertzuweisung eine entsprechende Fallunterscheidung einbauen, im Fall eines ungültigen negativen Werts als Fehlerwert `false` zurückgeben, und im Erfolgsfall `true` (und als Rückgabebetyp `boolean` statt `void` verwenden). Das war genau unser Vorgehen in der Programmiersprache C. In Java gibt es aber einen speziellen Mechanismus zur Ausnahme- und Fehlerbehandlung. Da wir diesen erst später einführen (Kapitel 6), verzichten wir bis dahin auf Überprüfungen der Eingabeparameter.

Mit dem dargestellten Konstruktor können wir `Person`-Objekte anlegen. Im Beispiel hat der Konstruktor Eingabeparameter für das Geburtsdatum und die Größe. Die für die Eingabeparameter übergebenen Werte werden dann im Konstruktor durch Aufruf der entsprechenden Setter in den Attributen gespeichert. Es fällt auf, dass wir hier keine direkten Wertzuweisungen an die Attribute verwenden, sondern stattdessen die Setter aufrufen. Dadurch können wir später die Setter noch anpassen, um zum Beispiel Datenstrukturinvarianten zu überprüfen, ohne die Konstruktoren nochmal verändern zu müssen (da wir diese Überprüfungen ja dort dann auch benötigen). Jetzt sieht man auch, warum es sinnvoll ist, die Methode `setDateOfBirth` anzulegen, obwohl sie privat ist. Denn innerhalb der Klasse können wir sie immer noch sinnvoll verwenden.

Da im Konstruktor keine Hobbys übergeben und gesetzt werden, hat ein `Person`-Objekt zu Beginn eine leere Liste von Hobbys. Um ein Hobby hinzuzufügen, muss man für ein `Person`-Objekt die `addHobby`-Methode aufrufen. An diese wird ein Hobby (als String) und die Position des Hobbys in der Liste aller Hobbys übergeben. Dabei wird der neue Wert nur gesetzt, wenn die übergebene Position auch in der Liste vorhanden ist. Entsprechend kann man mit `removeHobby` ein Hobby wieder löschen. Dabei wird der Wert an der entsprechenden Position auf `null` gesetzt. Der Wert `null` ist ein Standardwert für Objekte und bedeutet, dass kein Objekt (in diesem Fall: kein Hobby) vorhanden ist.

Jetzt können wir die Klasse `Person` in einer Programmklasse benutzen. Diese legen wir in einer separaten Datei im selben Paket an (für mehrere Pakete ist unser Programm noch zu klein). Zu beachten ist hierbei, dass wir in der Programmklasse nur öffentliche (Attribute und) Methoden der Klasse `Person` verwenden können. Diese bilden die Schnittstelle der Klasse `Person` im Sinne der Moduldefinition aus Kapitel 3. Der Code in `main` kann zum Beispiel wie folgt aussehen:

```

Person p;
p = new Person(LocalDate.of(1962, 1, 15), 181);
p.setSize(180);

```

```
p.addHobby("Reading", 0);
Person.setTax(19.0);
System.out.println(p);
```

Zuerst deklarieren wir eine Variable `p` vom Typ `Person`. Diese repräsentiert damit ein Objekt vom Typ `Person`. Dann rufen wir mit dem Schlüsselwort `new` einen Konstruktor der Klasse `Person` auf. Dadurch wird im Arbeitsspeicher dynamisch Speicherplatz für ein Objekt vom Typ `Person` reserviert. Dieser bietet Platz für die Aufnahme des **Inhalts** des Objekts – das sind die Datenwerte für alle Objektattribute (hier: ein `LocalDate`-Objekt und eine ganze Zahl). An den Konstruktor übergeben wir ein `LocalDate`-Objekt (das wir erst durch den Aufruf der Methode `of` erzeugen müssen) als Geburtsdatum und eine ganze Zahl als Größe, die dann im reservierten Speicherbereich abgelegt werden. Klassenattribute werden übrigens in einem anderen, zur Klasse gehörenden, Bereich des Arbeitsspeichers abgelegt.

Der Konstruktoraufruf liefert die Adresse des reservierten Speicherbereichs. Durch die Wertzuweisung an `p` wird diese Adresse in `p` gespeichert. Die Variable `p` ist also adresswertig. Der Speicherplatz für diese Adresse wird mit der Deklaration von `p` angelegt. Das trifft auf alle Variablen zu, die eine Klasse als Datentyp haben. Allgemein nennen wir solche Datentypen **Referenz-Datentypen**.

Nach dem Konstruktoraufruf können wir für das Objekt `p` unsere Objektmethoden verwenden. Für deren Aufruf müssen wir das Objekt `p` mit dem Punkt-Operator davorschreiben. Dadurch wird die Methode auf `p` angewendet. Wie wir vorher gesehen haben, müssen wir das nur bei Methodenaufrufen in einer anderen Klasse machen. Innerhalb derselben Klasse müssen wir nichts vor den Methodenaufruf setzen. Mit

```
p.setSize(180);
```

wird die bisherige Größe 181 von `p` mit 180 überschrieben. Mit

```
p.addHobby("Reading", 0);
```

wird das neue Hobby "Reading" an Position 0 in der Liste aller Hobbys eingetragen.

Für den Aufruf einer Klassenmethode müssen wir die Klasse `Person` mit dem Punkt-Operator davorschreiben. Mit

```
Person.setTax(19.0);
```

wird die bisherige Steuer 7.0 mit 19.0 überschrieben.

Durch die letzte Anweisung versuchen wir `p` mit `println` auszugeben, wie wir das schon mit Objekten von API-Klassen gemacht haben. Das Ergebnis sieht aber erstmal nicht sinnvoll aus, denn die Ausgabe hat in etwa die folgende Form:

```
Person@10FFA4...
```

Vor dem `@` wird der Klassenname ausgegeben, und hinter dem `@` die Speicheradresse des Objekts `p` in Hexadezimalform. Das ist die Standardausgabe von Objekten in Java in Form eines Strings. Wollen wir statt dieser Standardausgabe den Inhalt von `p` ausgeben, müssen wir das in einer separaten Methode `toString` (in der Klasse `Person`) definieren.

```

@Override
public String toString() {
    return "Geburtsdatum: " + getDateOfBirth() + "\n"
        + "Groesse: " + getSize() + "\n"
        + "Steuer: " + getTax() + "\n";
}

```

Die **Annotation** `@Override` vor der `toString`-Methode zeigt an, dass hier eine Methode aus einer anderen Klasse überschrieben wird. Was das bedeutet, klären wir etwas später. Wenn wir nun erneut die Ausgabe mit `println` ausprobieren, erhalten wir genau die von `toString` gelieferte Zeichenkette.

Den Hintergrund dafür wollen uns näher ansehen. Dabei steigen wir direkt in eines der fortgeschrittenen Themen der Objektorientierung ein – die **Vererbung**. Die Klassen der Java-API bilden nämlich eine sogenannte **Vererbungshierarchie**. Das bedeutet, dass sogenannte **Unterklassen** von sogenannten **Oberklassen** erben. In diesem Fall übernimmt die Unterklasse alle Attribute und Methoden der Oberklasse. Zusätzlich hat sie weitere spezifische Attribute und Methoden. Eine Unterklasse ist also eine Spezialisierung einer Oberklasse. Legt man ein Objekt einer Unterklasse an, kann man damit für dieses Objekt auch alle Methoden der Oberklasse verwenden. Zudem kann man ein Objekt einer Unterklasse für einen Oberklassentyp einsetzen. Denn ein Objekt einer Unterklasse **ist auch ein Objekt jeder Oberklasse** (es stehen ja alle Attribute und Methoden der Oberklasse zur Verfügung).

In der Vererbungshierarchie der Java-API gibt es eine sogenannte **Wurzelklasse**. Diese ist Oberklasse aller anderen Klassen und heißt `Object`. Auch wenn wir eine eigene Klasse wie `Person` anlegen, ist diese Unterklasse von `Object`. Die Klasse `Object` stellt damit Methoden bereit, die allen anderen Klassen zur Verfügung stehen. In UML stellt man Oberklassen wie in Abbildung 4 dar. Der Pfeil definiert `Person` als eine Unterklasse der Oberklasse `Object`. Vor den Klassennamen `Object` schreiben wir ihr API-Paket und machen dadurch kenntlich, dass es sich um eine API-Klasse handelt und nicht um eine selbst entworfene Klasse. Da wir die privaten Attribute von `Object` nicht kennen, geben wir keine an. Neben anderen Methoden hat `Object` die Methode `toString`. Diese wird grundsätzlich von der Unterklasse `Person` geerbt und wird dort deshalb zuerst einmal nicht angegeben. Außer wir wollen `toString` wie in unserem Fall in `Person` anders implementieren (das nennt man überschreiben). Dann schreiben wir `toString` auch noch einmal in die Klasse `Person`.

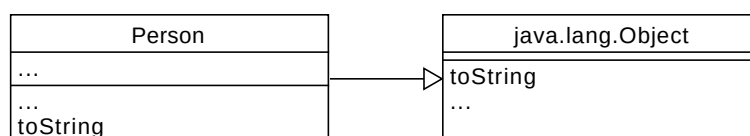


Abbildung 4: `Person` ist eine Unterklasse der Oberklasse `java.lang.Object` und überschreibt die Methode `toString`.

Die Methode `toString` ist in der Klasse `Object` so implementiert, dass sie eine Zeichenkette der Form


```
<Klassenname>@<Speicheradresse>
```

liefert. Rufen wir nun

```
System.out.println(p);
```

auf, so benutzen wir die folgende Variante von `println`:

```
public void println(Object x)
```

Das liegt daran, dass immer die zum Typ des übergebenen Arguments (hier: `p` vom Typ `Person`) passende Variante von `println` aufgerufen wird. Nun gibt es keine Variante von `println` mit einem Eingabeparameter vom Typ `Person`. Allerdings ist `Person` eine Spezialisierung von `Object`, `p` ist also auch vom Typ `Object`.

Die obige `println`-Variante ist nun wie folgt implementiert:

```
public void println(Object x) {  
    String s = String.valueOf(x);  
    print(s);  
    println();  
}
```

Das übergebene Objekt wird also mit der Klassenmethode `valueOf` der Klasse `String` in eine Zeichenkette umgewandelt, die dann ausgegeben wird. Die `valueOf`-Methode wiederum ruft `toString` des übergebenen Objekts auf:

```
public static String valueOf(Object obj) {  
    return obj.toString();  
}
```

Steht in der Klasse `Person` keine `toString`-Methode zur Verfügung, wird die `toString`-Methode der Oberklasse `Object` verwendet und wir bekommen eine nicht sonderlich sinnvolle Ausgabe. Stattdessen wird aber die Klasse `Person` selbst mit der `toString`-Methode aus, wird diese verwendet. In diesem Fall **überschreibt** die `toString`-Methode in `Person` die `toString`-Methode in `Object`. Deshalb versehen wir die Methode mit der **Annotation** `@Override`.

Bleibt die Frage, wieso die Implementierung der `toString`-Methode in `Object` nur eine sehr abstrakte Repräsentation des Inhalts eines Objekts (Klassenname und Speicheradresse) liefert. Die Antwort ist einfach: Mehr Informationen zum Inhalt stehen auf der abstrakten Ebene von `Object` nicht zur Verfügung – die Implementierung kann ja nur die Attribute der Klasse `Object` nutzen.

In den folgenden Kapiteln wollen wir nun formal und im Detail die verschiedenen Aspekte der Objektorientierung in Java definieren. Zur Illustration dienen die Java-API und die bisher betrachteten eigenen Datenklassen. Dabei werden wir die bisherigen Beispiele noch hier und da erweitern. Ziel ist es in erster Linie, die Java-API komplett verstehen zu können. Dabei werden wir auch schon sukzessive dazu passende UML-Notationen einführen.

4.2 Klassen und Objekte

Ein Java-Programm besteht normalerweise aus mehreren Klassen, und zwar aus

- einer Programmklasse mit der `main`-Methode,
- mehreren Datenklassen zur Verwaltung der Daten der Anwendung,
- Fensterklassen für die grafische Benutzerschnittstelle, und
- Klassen, die die Datenspeicherung übernehmen.

Bei der Implementierung dieser Klassen helfen uns zudem etliche vorgegebene Klassen der Java-API. Beispiele für Programm-, Daten- und API-Klassen haben wir bereits kennengelernt. Wir wollen Klassen nun ein erstes Mal allgemein definieren. Wir wollen die Definition dabei nicht auf Java beschränken, sondern sie so allgemein fassen, dass sie auch die programmiersprachenunabhängige Modellierung mit UML umfasst.

Definition 4.1: Klasse

Eine **Klasse** ist ein Schema zur Spezifikation gemeinsamer Eigenschaften mit deren Verwaltung einer Menge von gleichartigen Objekten (einer Objektart) und dient als **Datentyp**. Die durch eine Klasse vereinbarten Eigenschaften nennt man **Attribute**. Zur Verwaltung der Attribute gibt eine Klasse eine Menge von **Methoden** vor.

Eine Klasse repräsentiert also eine Menge gleichartiger Objekte. Wie wir bereits in einigen Beispielen gesehen haben, legen wir zur Laufzeit eines Programms solche Objekte an. Ein **Objekt** einer Klasse ist dabei eine konkrete Ausprägung einer Klasse.

Definition 4.2: Objekt

Ein **Objekt** einer Klasse ist ein zur Ausführungszeit des Programms vorhandenes **Exemplar** der Klasse (auch: eine **Instanz** der Klasse). Ein Objekt hat für jedes Attribut der Klasse einen konkreten Datenwert. Die Kombination dieser Datenwerte nennt man **Zustand** oder **Inhalt** des Objekts. Die Methoden der Klasse beschreiben das sogenannte **Verhalten** des Objekts. Es gelten folgende Prinzipien:

1. **(Objekt-Identitäts-Prinzip)**
Jedes Objekt besitzt eine von seinen Attributwerten unabhängige Identität.
2. **(Objekt-Klassen-Prinzip)**
Jedes Objekt gehört zu genau einer Klasse.

Das Objekt-Identitäts-Prinzip bedeutet, dass es zwei verschiedene Objekte (zum Beispiel Strings) geben kann, die den komplett gleichen Inhalt haben. Wir werden die beiden zuletzt genannten Prinzipien in den folgenden Kapiteln im Kontext von Java immer wieder aufgreifen und diskutieren.

4.2.1 Klassen in Java

Betrachten wir das Konzept von Klassen nun in Java. Man deklariert eine Klasse nach dem folgenden Schema. Dabei sind Schlüsselwörter in grün dargestellt. Optionale Angaben befinden sich in eckigen Klammern. Durch spitze Klammern werden Platzhalter angegeben.

Definition 4.3: Deklaration einer Klasse

```
[<Sichtbarkeit>] [abstract] [final] class <Klasse>  
    [extends <Oberklasse>]  
    [implements <Schnittstellenliste>]
```

Die Bedeutungen der einzelnen Teile sind dabei wie folgt definiert:

- <Sichtbarkeit>: Die **Sichtbarkeit** der Klasse.
- **abstract**: Definiert eine sogenannte **abstrakte Klasse** – siehe Kapitel 4.3.
- **final**: Definiert eine sogenannte **finale Klasse** – siehe Kapitel 4.3.
- <Klasse>: Der **Name** der Klasse.
- **extends** <Oberklasse>: Gibt an, von welcher Klasse die definierte Klasse erben soll – siehe Kapitel 4.3.
- **implements** <Schnittstellenliste>: Liste von sogenannten **Schnittstellen**, welche die Klasse **implementiert** (durch Kommas getrennt) – siehe Kapitel 4.4.

Definition 4.4: Sichtbarkeit von Klassen

Die häufigsten Sichtbarkeiten einer Klasse sind:

- **package private**: Definiert eine **paket-private** Klasse. Nur in Klassen, die sich im selben Paket wie die Klasse befinden, kann man auf die Klasse zugreifen. Das ist der Standardfall, wenn man keine Sichtbarkeit angibt.
- **public**: Definiert eine **öffentliche** Klasse. Auch in Klassen, die sich nicht im selben Paket wie die Klasse befinden, kann man auf die Klasse zugreifen.

Wie wir sehen, gibt es eine Menge Schlüsselwörter, die wir in der Definition unserer Klasse verwenden können – je nachdem, wie sich unsere Klasse verhalten soll. Hierbei sind einige Angaben optional, allerdings muss die Reihenfolge der Schlüsselwörter und Platzhalter eingehalten werden. Die kürzeste Deklaration einer Klasse sieht wie folgt aus:

```
class <Klasse>
```

Wir wollen am Beispiel der Klasse `String` der API sehen, was eine elaboriertere Klassen-Deklaration konkret zu bedeuten hat.

Beispiel 4.5: Eigenschaften der Klasse String aus der Java-API

Die API-Klasse

```
public final class String extends Object implements
    Serializable, CharSequence, Comparable<String>
```

- ist eine **öffentliche** Klasse. Deswegen können wir in unseren eigenen Programmen problemlos die Klasse `String` verwenden.
- ist **nicht abstrakt**
- ist **final**
- **erbt von** `Object`
- implementiert die **Schnittstellen** `Serializable`, `CharSequence` und `Comparable<String>`

Die Deklaration einer Klasse zusammen mit ihrem **Rumpf** ergibt die Definition der Klasse. Der Rumpf schließt sich an die Deklaration an und befindet sich innerhalb geschweifter Klammern. Im Rumpf befinden sich alle Angaben zu Attributen, Konstruktoren und Methoden zur Erstellung und Verwaltung von Objekten der Klasse.

Definition 4.6: Definition einer Klasse

```
[<Sichtbarkeit>] [abstract] [final] class <Klasse>
    [extends <Oberklasse>]
    [implements <Schnittstellenliste>] {

    <Attributliste>
    <Konstruktorliste>
    <Methodenliste>
}
```

In den folgenden Abschnitten werden wir die Bestandteile des Rumpfs genauer betrachten und definieren.

4.2.2 Attributliste

Attribute einer Klasse definieren die Eigenschaften der Objekte der Klasse. Ein Attribut ist eine Variable, die in der gesamten Klasse gültig ist, wobei man zwischen **Objektattributen**, **Klassenattributen** und **Klassenkonstanten** unterscheidet. Eine Klasse bildet einen sogenannten **Namensraum** für Attributnamen, was bedeutet, dass die Attributnamen innerhalb einer Klasse eindeutig sein müssen.

Definition 4.7: Deklaration eines Objektattributs

Ein **Objektattribut** definiert eine Eigenschaft von Objekten, für die die Objekte der Klasse **unterschiedliche** Werte haben können. Es wird wie folgt deklariert:

```
[<Sichtbarkeit>] <Typ> <Attribut> [= <Ausdruck>];
```

Die Bedeutungen der einzelnen Teile sind dabei wie folgt definiert:

- <Sichtbarkeit>: **Sichtbarkeit** des Attributs.
- <Typ>: Der **Datentyp** des Attributs. Dieser kann ein primitiver Datentyp, eine Klasse oder ein Array sein.
- <Attribut>: Der **Name** des Attributs.
- = <Ausdruck>: **Wertzuweisung** für das Attribut.

Ohne explizite Wertzuweisung haben Attribute in Java standardmäßig den zum Datentyp gehörenden Null-Wert. Im Fall von Objekten ist dies der Standardwert null (dazu im nächsten Abschnitt mehr).

Der Zugriff auf ein Objektattribut **innerhalb der Klasse** erfolgt mit dem Attributenamen <Attribut>. Existieren lokale Variablen mit demselben Namen, so verwendet man `this.<Attribut>`.

Als Beispiel für ein Objektattribut haben wir bereits das Attribut `name` der Klasse `Person` in der Einleitung kennengelernt. Dieses Attribut hat für jede konkrete Person einen konkreten Wert, der zwar gleich sein kann, aber nicht gleich sein muss. So kann es Personen mit dem Namen Horst und Personen mit dem Namen Sophie geben.

In der API finden wir nur sehr wenige Beispiele für Objektattribute, denn die API-Dokumentation einer Klasse enthält nur die Beschreibungen aller öffentlichen Attribute (*Field Summary*). Die Objektattribute der API-Klassen sind in der Regel privat, um sie nach dem Einkapselungsprinzip vor dem direkten Zugriff von außen zu schützen. Auf ihre Werte kann nur über öffentliche Verwaltungsmethoden zugegriffen werden.

Definition 4.8: Deklaration eines Klassenattributs

Ein **Klassenattribut** repräsentiert eine Information über die Klasse selbst. Es wird so definiert:

```
<Sichtbarkeit> static <Typ> <Attribut> [= <Ausdruck>];
```

Die Deklaration unterscheidet sich von Objektattributen nur durch das Schlüsselwort `static`.

Der Wert eines Klassenattributs ist **eindeutig** für die Klasse und unabhängig von deren Objekten. Es existiert auch, wenn es (noch) keine Objekte der Klasse gibt.

Der Zugriff auf ein Klassenattribut **innerhalb der Klasse** erfolgt mit dem Attributenamen `<Attribut>`. Existieren lokale Variablen mit demselben Namen, so verwendet man `<Klasse>.<Attribut>`.

Definition 4.9: Sichtbarkeit von Attributen

Mögliche Sichtbarkeiten sind:

- `private`: Definiert ein **privates** Attribut. Zugriff auf das Attribut ist nur innerhalb der Klasse möglich (**Standardfall** bei Attributen).
- `public`: Definiert ein **öffentliches** Attribut. Zugriff auf das Attribut ist auch außerhalb der Klasse möglich.
- `protected`: Definiert ein **geschütztes** Attribut. Nur in Klassen aus demselben Paket wie diese Klasse, sowie in Klassen, die von dieser Klasse erben (gegebenenfalls aus anderen Paketen), kann man auf das Attribut zugreifen.
- Ist keine Sichtbarkeit angegeben, so kann man in allen Klassen desselben Pakets auf das Attribut zugreifen.

Zugriff auf ein **nicht privates** Objektattribut eines Objekts `o` **in anderen Klassen**:

```
o.<Attribut>
```

Zugriff auf ein **nicht privates** Klassenattribut einer Klasse `K` **in anderen Klassen**:

```
K.<Attribut>
```

Als Beispiel für ein Klassenattribut haben wir bereits das Attribut `tax` der Klasse `Person` in der Einleitung kennengelernt. Dieses Attribut hat für alle Personen den gleichen Wert.

Der Wert von Objekt- und Klassenattributen kann sich zur Laufzeit des Programms ändern, zum Beispiel wenn eine Person mit dem Namen Horst ihren Namen in Heinrich ändert. Konstanten sind Attribute, deren Wert sich nicht ändern kann.

Definition 4.10: Deklaration und Benutzung einer Klassenkonstante

Eine **Klassenkonstante** ist ein Klassenattribut, dessen Wert sich zur Laufzeit des Programms nicht ändern kann.

```
<Sichtbarkeit> static final <Typ> <Attribut> [= <
    Ausdruck>];
```

Die Deklaration unterscheidet sich von Klassenattributen nur durch das Schlüsselwort `final`.

Klassenkonstanten sind in der Regel öffentlich. Einer Klassenkonstante kann genau einmal ein Wert zugewiesen werden.

Beispiele für öffentliche Klassenkonstanten finden wir in der Klasse `Math` aus der API. Diese stellt unter anderem die numerischen Konstanten

- `public static final double E`: Die eulersche Zahl e
- `public static final double PI`: Die Kreiszahl π

als Klassenkonstanten zur Verfügung. In unserem grafischen Editor haben wir in Kapitel 2 für die verschiedenen möglichen Randfarben private Klassenkonstanten eingeführt.

4.2.3 Konstruktorliste

Konstruktor sind Methoden zur erstmaligen Initialisierung der Attribute eines erstellten Objekts. Die Form des Konstruktors ist dabei eindeutig festgelegt: Er muss genau so heißen, wie die Klasse selbst und darf keinen Rückgabetyt haben. Wird ein Konstruktor aufgerufen, soll sich dieser darum kümmern, ein Objekt zu erzeugen und die Attribute des erzeugten Objekts mit den übergebenen Werten zu initialisieren. Die eigentliche Erzeugung im Sinne von Speicherreservierung nimmt Java automatisch vor, die Zuweisung der Attributswerte müssen wir explizit im Konstruktor durchführen.

Definition 4.11: Deklaration eines Konstruktors

Ein Konstruktor wird in Java wie folgt deklariert:

```
[ <Sichtbarkeit> ] <Konstruktor> ( <Parameterliste> ) [
    throws <Ausnahmenliste> ]
```

Hierbei bedeuten:

- <Sichtbarkeit>: Die **Sichtbarkeit** des Konstruktors.
- <Konstruktor>: Der **Name** des Konstruktors. Dieser entspricht dem Klassennamen.
- <Parameterliste>: Die Liste der **Eingabeparameter**. Für diese werden bei Aufruf Werte für die Attribute übergeben. Die Eingabeparameter heißen dabei in der Regel genauso wie die Attribute.
- <Ausnahmenliste>: Die Liste der **Ausnahmen**, die bei Ausführung des Konstruktors auftreten können – mehr dazu in Kapitel 6.

Ist die Parameterliste leer, spricht man von einem **leeren Konstruktor**.

Definition 4.12: Sichtbarkeit von Konstruktoren

Mögliche Sichtbarkeiten sind:

- **private**: Der Konstruktor ist **privat**. Ein Aufruf ist nur innerhalb der Klasse möglich.
- **public**: Der Konstruktor ist **öffentlich**. Ein Aufruf ist auch außerhalb der Klasse möglich. Das ist der **Standardfall**.
- **protected**: Der Konstruktor ist **geschützt**. Ein Aufruf ist innerhalb des Pakets möglich, sowie in Klassen außerhalb des Pakets, die von dieser Klasse erben.
- Lässt man die Sichtbarkeit des Konstruktors weg, so ist diese **package private**, sodass nur Klassen innerhalb desselben Pakets den Konstruktor aufrufen können.

Ein Konstruktor muss nicht für jedes Attribut einen Eingabeparameter besitzen. Sieht man für ein bestimmtes Attribut keinen Eingabeparameter vor, so erhält das Attribut den Standardwert. Der leere Konstruktor setzt also alle Attribute auf den Standardwert. Wird lediglich ein leerer Konstruktor benötigt, so muss dieser nicht ausdrücklich angegeben werden, da er vom Compiler automatisch erzeugt wird, wenn er fehlt. Allerdings ist dies nur der Fall, solange kein anderer Konstruktor der Klasse definiert wurde. Eine Klasse kann einen oder mehrere Konstruktoren mit unterschiedlichen Parameterlisten besitzen. Diese Konstruktoren führen dann unterschiedliche Initialisierungen der Attribute durch.

In der Einleitung haben wir bereits den folgenden Konstruktor der Klasse `Person` erstellt und benutzt:

```
public Person(LocalDate dateOfBirth, int size)
```


Erstellen wir ein Objekt vom Typ `Person` mit diesem Konstruktor, so müssen wir bei Aufruf das Geburtsdatum und die Grösse an den Konstruktor übergeben. Wir erhalten dadurch ein Objekt, dessen Attribute genau so belegt sind, wie wir es übergeben haben. Ein weiterer möglicher Konstruktor wäre zum Beispiel:

```
public Person(int size)
```

In diesem Fall müssen wir nur die Größe der Person übergeben. Das Geburtsdatum wird automatisch auf den Standardwert `null` gesetzt. Dieser steht für einen unbekannten Wert.

Die Deklaration eines Konstruktors zusammen mit seinem **Rumpf** ergibt die Definition des Konstruktors. Der Rumpf schließt sich an die Deklaration an und befindet sich innerhalb geschweifter Klammern. Im Rumpf befinden sich die Anweisungen zum Initialisieren der Attribute mit den übergebenen Werten. Dafür benutzen wir grundsätzlich für jedes Attribut eine **Set-Methode** (siehe nächster Abschnitt).

Definition 4.13: Definition eines Konstruktors

```
<Sichtbarkeit> <Konstruktor> (<Parameterliste>) {  
  
    //Initialisierung der Attribute durch Set-  
        Methoden  
  
}
```

Für den ersten Konstruktor der Klasse `Person` wird seine Implementierung in der Einleitung gezeigt. Für den zusätzlich oben eingeführten Konstruktor sieht die Implementierung wie folgt aus:

```
public Person(int size) {  
    setSize(size);  
}
```

Der Standardwert für das Attribut `dateOfBirth` müssen wir nicht selbst explizit setzen – das erfolgt automatisch.

In der API lassen sich die Konstrukturen einer Klasse unter *Constructor Summary* finden. So können wir leicht entdecken, dass die Klasse `String` eine Vielzahl von Konstruktoren anbietet, darunter auch einen leeren Konstruktor

```
public String()
```

Dieser erzeugt laut Beschreibung einen leeren String (das ist ein String der Länge 0). Daneben gibt es zum Beispiel den Kopierkonstruktor

```
public String(String original)
```

Dieser erzeugt einen neuen String mit demselben Inhalt wie der String `original`. Nicht alle API-Klassen haben übrigens öffentliche Konstrukturen, wie zum Beispiel `LocalDate`. Solche Klassen bieten stattdessen Klassenmethoden zur Erzeugung von Objekten an. Als Beispiel haben wir bereits die Methode `of` der Klasse `LocalDate` kennengelernt. Formal werden Klassenmethoden im nächsten Abschnitt eingeführt.

Unseren neuen Konstruktor der Klasse `Person` können wir wie folgt verwenden:

```
Person p;  
p = new Person(181);
```

Dadurch wird ein `Person`-Objekt `p` angelegt mit Grösse 181 und Geburtsdatum `null`. Allgemein gilt:

Definition 4.14: Aufruf eines Konstruktors

Durch einen **Konstruktoraufruf** der Form

```
o = new <Konstruktor>(<Werteliste>);
```

wird ein Objekt erzeugt und Attribute des erzeugten Objekts mit den übergebenen Werten initialisiert. Attribute, für die kein Eingabeparameter vorgesehen ist, kann man einen einheitlichen Anfangswert zuweisen oder erhalten den Standardwert ihres Datentyps. Das erzeugte Objekt wird dann `o` zugewiesen. Hierdurch wird `o` **initialisiert**.

4.2.4 Methodenliste

Eine Methode dient zur Verwaltung von Attributen, zur Erzeugung von Objekten oder zu Berechnungen. Wir unterscheiden zwischen **Objekt-** und **Klassenmethoden**.

Definition 4.15: Deklaration einer Objektmethode

Der **Kopf** einer Objektmethode wird wie folgt deklariert:

```
[<Sichtbarkeit>] [abstract] [final] <Typ> <Methode>  
(<Parameterliste>)  
[throws <Ausnahmenliste>]
```

Dabei sind die einzelnen Bestandteile wie folgt definiert:

- `<Sichtbarkeit>`: Die **Sichtbarkeit** der Methode.
- `abstract`: Die Methode ist **abstrakt** – mehr dazu in Kapitel 4.3.
- `final`: Die Methode ist **final** – mehr dazu in Kapitel 4.2.
- `<Typ>`: Der **Rückgabotyp** der Methode oder `void`, falls kein Wert zurückgegeben wird.
- `<Methode>`: Der **Name** der Methode.
- `<Parameterliste>`: Die Liste der **Eingabeparameter** der Methode.
- `<Ausnahmenliste>`: Die Liste der **Ausnahmen**, die bei Ausführung der Methode auftreten können – mehr dazu in Kapitel 6.

Die Deklaration einer Methode zusammen mit ihrem **Rumpf** ergibt die Definition der Methode. Der Rumpf schließt sich an die Deklaration an und befindet sich innerhalb geschweiften Klammern. Im Rumpf befinden sich die Anweisungen, die bei Aufruf der Methode ausgeführt werden.

Klassische Objektmethoden sind Get- und Set-Methoden für den elementaren Zugriff auf private Objektattribute:

- **Get-Methode:**

Liefert einen Attributwert. Übliche Implementierung:

```
public <Typ> get<Attribut>() {  
    return <Attribut>;  
}
```

- **Set-Methode:**

Überschreibt einen aktuellen Attributwert durch den übergebenen Wert. Erste einfache Implementierung (ohne Überprüfung des neuen Werts auf Gültigkeit):

```
public void set<Attribut>(<Typ> <Attribut>) {  
    this.<Attribut> = <Attribut>;  
}
```

In der Einleitung haben wir bereits Beispiele für die Implementierung und den Aufruf solcher Methoden gesehen. Ist ein Attribut ein Array und verwaltet auf diese Weise mehrere Werte, ist es auch üblich, die zugehörigen Zugriffsoperationen anders zu nennen:

- **Add-Methode** (für Arrays):

Fügt dem Array einen übergebenen Wert hinzu. Eine mögliche einfache Implementierung wäre (ohne Überprüfung des neuen Werts auf Gültigkeit):

```
public void add<Attribut>(<Typ> <Attribut>, int position) {  
    if (position < this.<Attribut>.length)  
        this.<Attribut>[position] = <Attribut>;  
}
```

Hier wird der neue Wert nur gesetzt, wenn die übergebene Position auch im Array vorhanden ist.

- **Remove-Methode** (für Arrays):

Löscht einen Wert aus dem Array an der übergebenen Position, indem an dieser Position der Standardwert des Datentyps eingetragen wird. Erste einfache Implementierung:

```
public void remove<Attribut>(int position) {  
    if (position < this.<Attribut>.length)  
        this.<Attribut>[position] = <Standardwert>;  
}
```

Für unsere eigenen Klassen wollen wir uns durchweg an diese Namenskonventionen halten, da sie die Implementierung im Kontext einer konkreten Anwendung besser lesbar und wartbar machen.

Auch die Java-API enthält viele Beispiele für Objektmethoden, die wir bereits kennengelernt haben. Manche von diesen sind ebenfalls zum Lesen oder zum Schreiben von Attributwerten da, benutzen aber teilweise andere Namenskonventionen. Andere testen zum Beispiel Eigenschaften eines Objekts. Beispiele sind:

- Klasse `String`: `charAt` (Lesemethode), `length` (Lesemethode), `concat` (Schreibmethode), `replace` (Schreibmethode), `isEmpty` (Testmethode).

- Klasse `LocalDate`: `getYear` (Lesemethode), `minusDays` (Schreibmethode).

Definition 4.16: Deklaration einer Klassenmethode

Der **Kopf** einer Klassenmethode wird wie folgt deklariert:

```
[<Sichtbarkeit>] static [final] <Typ> <Methode>
    (<Parameterliste>)
    [throws <Ausnahmenliste>]
```

Klassenmethoden unterscheiden sich von Objektmethoden nur durch das Schlüsselwort `static`.

Eine Klassenmethode definiert eine Methode, für deren Aufruf kein Objekt benötigt wird, sondern die direkt durch Benutzung des Klassennamens aufgerufen werden kann.

Definition 4.17: Sichtbarkeit und Aufruf von Methoden

Mögliche Sichtbarkeiten sind:

- `private`: Die Methode ist **privat**. Ein Aufruf der Methode ist nur innerhalb der Klasse möglich.
- `public`: Die Methode ist **öffentlich**. Ein Aufruf der Methode ist auch außerhalb der Klasse möglich. Dies ist der **Standardfall**.
- `protected`: Die Methode ist **geschützt**. Ein Aufruf der Methode ist innerhalb des Pakets möglich, sowie in Klassen außerhalb des Pakets, die von dieser Klasse erben.
- `package private`: Die Methode ist **paket-privat**. Ein Aufruf der Methode ist außerhalb der Klasse, aber nur innerhalb des Pakets dieser Klasse möglich.

Für ein Objekt `o` führt folgender Aufruf eine **nicht private** Objektmethode auf den Attributwerten von `o` aus:

```
o.<Methode>(<Werteliste>)
```

Für eine Klasse `K` führt folgender Aufruf eine **nicht private** Klassenmethode aus:

```
K.<Methode>(<Werteliste>)
```

Eine Anwendung von Klassenmethoden ist die Verwaltung von Klassenattributen, zum Beispiel des Attributs `tax` der Klasse `Person` mit zugehörigen Get- und Set-Methoden (die diesmal Klassenmethoden sind).

Die Java-API enthält ebenfalls viele Beispiele für Klassenmethoden, die wir bereits kennengelernt haben. Diese sind zum Beispiel für nicht objektbezogene Berechnungen / mathematische Funktionen und für die Erstellung von Objekten da. Beispiele sind:

- Klasse `Math`: `sqrt` (Berechnungsmethode), `pow` (Berechnungsmethode).
- Klasse `LocalDate`: `of` (Objekterstellungsmethode), `now` (Objekterstellungsmethode).
- Klasse `String`: `valueOf` (Objekterstellungsmethode).

Wie bei Konstruktoren kann es sinnvoll sein, mehrere Methoden mit demselben Namen, aber mit unterschiedlichen Parameterlisten zur Verfügung zu stellen. Dies ist in Java möglich und wird als **Überladen** von Methoden bezeichnet.

Definition 4.18: Überladene Methoden

Im Gegensatz zu Attributnamen müssen Methodennamen nicht eindeutig sein. Zwei (oder mehr) Methoden mit demselben Namen nennt man **überladen**. Sie müssen sich allerdings durch die Parameterliste unterscheiden, sodass bei Aufruf anhand der übergebenen Werte entschieden werden kann, welche Methode ausgeführt werden soll.

Tatsächlich haben wir schon häufig mit überladenen Methoden und Operatoren gearbeitet, ohne dies aktiv zu bemerken. So ist beispielsweise auch der Operator `+` überladen, denn wir können Zahlen unterschiedlicher Art miteinander addieren: Der Compiler erkennt automatisch, ob die Eingaben zum Beispiel vom Typ `int` oder `double` sind und gibt das Ergebnis im gleichen Typ zurück. In Java können wir darüber hinaus, wie in Kapitel 2 demonstriert, Zeichenketten mit `+` aneinander hängen.

Davon abgesehen gibt es aber noch viele weitere überladene Methoden in der API. Ein Beispiel ist die Methode `println` der Klasse `PrintStream`, die es dort gleich in 10 Varianten gibt. Drei Beispiele sind:

- `void println(int x)`: eine ganze Zahl ausgeben.
- `void println(double x)`: eine Dezimalzahl ausgeben.
- `void println(String x)`: einen String ausgeben.

Dadurch ist es möglich, Werte ganz unterschiedlichen Datentyps auszugeben. Auch andere in dieser Klasse vorhandene Methoden wie `append`, `format`, `print`, `printf` und `write` sind überladen. Ein ähnliches Beispiel ist die `valueOf`-Methode der Klasse `String`.

Ein konkretes Beispiel für eine Klasse inklusive Objektattributen, Klassenattributen, überladenen Konstruktoren und Verwaltungsmethoden finden Sie in *Screencast 03: Einfach Klasse! – Einfache Klassen in Java*.

4.2.5 Objekte in Java

Die Speicherverwaltung von Variablen mit primitivem Datentyp in Java (siehe Kapitel 2 für eine Liste aller primitiven Datentypen in Java) entspricht derjenigen in C. Mit Deklaration einer solchen Variable `v`

```
<Typ> v;
```

wird automatisch ausreichend Speicherplatz zur Aufnahme eines Werts vom entsprechenden Datentyp in einem bestimmten Bereich des Arbeitsspeichers angelegt, dem sogenannten **Stack**. Bei Wertzuweisungen an `v`

```
v = <Wert>;
```

wird dort der zugewiesene Wert abgelegt, und über den Variablennamen `v` kann man auf den dort abgelegten Wert zugreifen.

Neben primitiven Datentypen gibt es in Java **Referenz-Datentypen**. Zu den Referenz-Datentypen zählen Klassen und Arrays (Arrays werden in einem späteren Unterkapitel genauer erklärt). Zum Beispiel sind `String`, `LocalDate` und `Person` Referenz-Datentypen.

Definition 4.19: Referenz-Datentypen und Objekte

Alle Datentypen, die nicht primitive Datentypen sind, sind **Referenz-Datentypen**. Dazu zählen **Klassen** und **Arrays**. Variablen mit einem Referenz-Datentyp nennt man in Java **Objekte**.

Die Speicherverwaltung von Objekten unterscheidet sich fundamental von der Speicherverwaltung von Variablen mit primitivem Datentyp. Mit Deklaration eines Objekts `o`

```
<Typ> o;
```

wird automatisch ausreichend Speicherplatz im Stack zu Aufnahme einer **Speicheradresse** angelegt. Dort wird also nicht der Inhalt gespeichert. Um `o` benutzen zu können, muss man vielmehr erst noch Speicherplatz für den **Inhalt** des Objekts reservieren. Das kann man mit einem **Konstruktoraufruf** machen:

```
o = new <Konstruktor>(<Werteliste>);
```

Dieser sorgt für die Reservierung von ausreichend Speicherplatz zu Aufnahme des Inhalts von `o` in einem anderen Bereich des Arbeitsspeichers, dem sogenannten **Heap**. Die Speicheradresse dieses neuen Speicherbereichs im Heap wird vom Konstruktoraufruf zurückgegeben und durch die Wertzuweisung in `o` gespeichert. Jetzt passt alles zusammen, denn `o` ist ja adresswertig. Ein Objekt `o` ist also eine Referenz auf einen Speicherbereich im Heap, wo dessen Inhalt gespeichert ist. Das kann man sich so ähnlich vorstellen wie bei einem Weblink, der eine Webseite referenziert. Damit ist jetzt auch klar, warum man solche Datentypen als Referenz-Datentypen bezeichnet. Denn ein Objekt ist technisch eine Referenz (auf den Speicherbereich mit seinem Inhalt). Mit einer Deklaration

```
String s;
```

repräsentiert `s` also erstmal nur eine Speicheradresse. Nach dem Konstruktoraufruf

```
s = new String("Hello");
```

referenziert `s` einen Speicherbereich im Heap mit dem Inhalt `"Hello"`. Deklaration und Konstruktoraufruf kann man auch in eine einzige Anweisung schreiben.

Alternativ zu Konstruktoraufrufen gibt es statische Methoden, mit denen man Speicherplatz für den Inhalt von Objekten reservieren und an diesem Attributwerte speichern kann:

```
String t = String.valueOf(10.9);
```

Hier referenziert `t` einen Speicherbereich mit Inhalt `"10.9"`. Hat eine Methode also einen Referenz-Datentyp als Rückgabetyp, so gibt sie eine Speicheradresse zurück. Diese weist man einem Objekt (passenden Typs) als Wert zu und kann so auf den Inhalt an dieser Speicheradresse zugreifen.

Definition 4.20: Objekte – Deklaration und Initialisierung neuer Objekte

Objekte sind **adresswertig** und **referenzieren** den Speicherbereich, in dem ihr Inhalt gespeichert ist. Bei der **Deklaration**

```
<Typ> o;
```

eines Objekts `o` wird **auf dem Stack Speicherplatz für eine Speicheradresse** reserviert. Bevor man `o` benutzen kann, muss man `o` zuerst **initialisieren**. Das kann man über den Aufruf eines Konstruktors der Form

```
o = new <Konstruktor>(<Werteliste>);
```

machen. Dadurch wird **auf dem Heap ein Speicherbereich für den Inhalt von `o`** geschaffen und `o` die Adresse dieses Speicherbereichs zugewiesen. Alternativ kann man `o` wie folgt mit dem Aufruf einer Klassenmethode, die eine Referenz auf einen Inhalt passenden Typs liefert, initialisieren:

```
o = <Klasse>.<Methode>(<Werteliste>);
```

Auf den Inhalt eines Objekts können wir mit Aufruf von Objektmethoden des Datentyps des Objekts zugreifen. Wie solche Methodenaufrufe aussehen, haben wir schon besprochen. Das können unter anderem lesende und schreibende Methoden sein.

Merke 4.21: Objekte – Objektmethoden

Auf den Inhalt eines Objekts `o` greift man wie folgt durch **Aufruf einer Objektmethode** `Methode` des Datentyps des Objekts zu:

```
o.<Methode>(<Werteliste>);
```

Mit schreibenden Methoden können wir den Inhalt eines Objekts ändern. Hierbei müssen wir nur aufpassen, ob von einer Objektmethode tatsächlich der Inhalt des Objekts überschrieben wird, oder eine Kopie des Objekts mit geändertem Inhalt angelegt wird. Zum Beispiel ändert die Methode `setSize` im Code

```
Person p = new Person(LocalDate.of(1962, 1, 15), 181);  
p.setSize(180);
```

den Wert des Attributs `size` des Inhalts von `p` von 181 auf 180. Andererseits legt die Methode `replace`, wie wir in der API nachlesen können, im Code

```
String t = String.valueOf(10.9);
t.replace('.', '-');
```

an einer neuen Speicheradresse eine Kopie des Inhalts "10.9" von `t` an und ändert die Kopie in "10-9". Der Inhalt von `t` ändert sich hier also gar nicht. Und wie bekommen wir nun Zugriff auf die Kopie mit dem geänderten Inhalt? Ganz einfach: `replace` liefert die Speicheradresse der Kopie, die wir über eine Wertzuweisung speichern können. Durch

```
t = t.replace('.', '-');
```

speichern wir die Speicheradresse der Kopie in `t`, so dass `t` nun eine Referenz auf die (geänderte) Kopie ist (also als neuen Inhalt die Kopie hat). Wir können den Inhalt eines Objekts also auch durch eine Wertzuweisung einer neuen Adresse ändern. Bei Wertzuweisungen an Objekte muss man nur darauf achten, dass auf der rechten Seite auch eine Referenz auf einen Inhalt passenden Typs steht.

Merke 4.22: Objekte – Referenzen als Rückgabotyp von Methoden

Ist der Rückgabotyp einer Methode ein Referenz-Datentyp, so gibt die Methode eine Speicheradresse zurück. Diese können wir über eine Wertzuweisung in einem Objekt speichern und erhalten so Zugriff auf den Inhalt an dieser Speicheradresse.

Definition 4.23: Objekte – Wertzuweisung

Eine **Wertzuweisung** an ein Objekt `o` hat die Form

```
o = <Referenz>;
```

wobei `<Referenz>` eine Referenz auf einen Speicherbereich ist, dessen Inhalt denselben Typ wie `o` hat. Solche Referenzen erhält man zum Beispiel über den Aufruf eines Konstruktors, den Aufruf einer Methode, die eine solche Referenz liefert, oder auch über ein anderes Objekt desselben Typs. Durch eine Wertzuweisung ändert sich der Speicherbereich, den ein Objekt referenziert.

Da im obigen Beispiel `s` und `t` beide Zeichenketten referenzieren, können wir zum Beispiel auch

```
s = t;
```

schreiben. Durch diese Anweisung referenziert `s` denselben Speicherbereich (und damit auch den identischen Inhalt) wie `t` und der Ausdruck `s == t` hat den Wert `true`. In diesem Fall bezeichnen wir `s` und `t` als **identische Objekte**.

Davon unterscheiden wir **gleiche Objekte**. Das sind Objekte, die unterschiedliche Speicherbereiche referenzieren (weshalb der Ausdruck `s == t` den Wert `false` hat), in denen aber der gleiche Inhalt steht. In folgendem Beispiel sind `s` und `t` **gleich, aber nicht identisch**:

```
String s = new String("Hello");
String t = new String("Hello");
```


Das **Objekt-Identitäts-Prinzip** wird in Java also über Speicheradressen umgesetzt.

Definition 4.24: Objekte – Identität und Gleichheit

Zwei Objekte `o1` und `o2` heißen **identisch**, wenn der Ausdruck `o1 == o2` den Wert `true` hat.

Zwei Objekte `o1` und `o2` heißen **gleich**, wenn sie den gleichen Inhalt haben.

Es gilt:

- Identische Objekte sind gleich.
- Gleiche Objekte haben den gleichen Typ.

Ein spezieller Adresswert ist `null`. Dieser Wert repräsentiert eine **unbekannte Referenz**.

Definition 4.25: Objekte – unbekannte Referenz

Der spezielle Adresswert `null` bedeutet **unbekannte Referenz** und hat verschiedene Anwendungen:

- `null` ist der **Standardwert von Attributen mit einem Referenz-Datentyp**.
- Methoden mit einem Referenz-Datentyp als Rückgabetyt liefern üblicherweise den Wert `null`, wenn bei deren Abarbeitung ein Fehler aufgetreten ist.
- Wird ein Objekt nicht mehr benötigt, weist man ihm den Wert `null` zu.

Für ein Objekt mit dem Wert `null` lassen sich keine Objektmethoden aufrufen, denn es ist ja kein Inhalt vorhanden, auf den diese angewendet werden könnten. Der Versuch eines Methodenaufrufs für ein solches Objekt führt zum Programmabbruch.

Java bietet eine automatisierte Freigabe nicht mehr verwendeter Daten im Heap, die sogenannte **Garbage Collection**.

Definition 4.26: Garbage Collection

Nicht mehr benutzte Speicherbereiche im Heap werden von Java **automatisch** wieder freigegeben. Diesen Prozess nennt man **Garbage Collection**. Dabei werden alle Speicherbereiche freigegeben, auf die keine Referenz mehr existiert.

In Java weist man dazu einem nicht mehr benötigten Objekt den Wert `null` zu – dadurch referenziert es keinen Speicherbereich mehr und dieser kann von der Garbage Collection freigegeben werden (falls es nicht noch weitere Referenzen auf denselben Speicherbereich gibt).

Nach folgenden Anweisungen

```
String s = new String("Hello");
String t = String.valueOf(10.9);
s = t;
```

gibt es keine Referenz mehr auf den Speicherbereich mit dem Inhalt "Hello", da `s` nach der letzten Anweisung einen anderen Speicherbereich referenziert (nämlich denselben wie `t` mit dem Inhalt "10.9"). Damit gibt die Garbage Collection den Speicherbereich mit dem Inhalt "Hello" wieder frei.

Hat ein Eingabeparameter einer Methode einen primitiven Datentyp, und wird bei Methodenaufruf ein Wert für diesen Eingabeparameter übergeben, so wird für diesen Wert eine Kopie auf dem Stack angelegt. In der Methode wird dann mit dieser Kopie gerechnet. Diese Kopie existiert, solange die Methode abgearbeitet wird, danach wird der Speicherplatz (für die Kopie) wieder freigegeben. Das bedeutet, dass man in einer Methode den Wert einer übergebenen Variable mit primitivem Datentyp nicht ändern kann, da alle Änderungen ja nur auf einer Kopie erfolgen. Dieses Prinzip nennt man *Call by Value*-Prinzip. Für Objekte gilt hingegen das *Call by Reference*-Prinzip.

Definition 4.27: Objekte – Call by Reference

Hat ein Eingabeparameter einer Methode einen Referenz-Datentyp, so wird bei Methodenaufruf für diesen Eingabeparameter eine Speicheradresse übergeben. In der Methode wird dann mit einer (Kopie der) Referenz auf den adressierten Speicherbereich gerechnet. Über diese Referenz lässt sich in der Methode mit Objektmethoden auf den Inhalt eines übergebenen Objekts zugreifen. Dieses Prinzip nennt man **Call by Reference**-Prinzip.

Jetzt können wir uns noch einmal genauer folgende Implementierung der `getPosition`-Methode aus Kapitel 2 ansehen:

```
public Point getPosition() {
    return new Point(position);
}
```

Nehmen wir an, wir hätten stattdessen folgende Implementierung gewählt:

```
public Point getPosition() {
    return this.position;
}
```

Das hätte unerwünschte Folgen auf den Kopierkonstruktor

```
public Circle(Circle original) {
    this.setPosition(original.getPosition());
    this.setRadius(original.getRadius());
}
```

In diesem Fall wird mit `original.getPosition()` eine Referenz auf die Position des Originals an die Kopie übergeben. Damit sind in Original und Kopie Referenzen auf das identische `Point`-Objekt

gespeichert. Wird der Inhalt dieses `Point`-Objekts über eine der Referenzen geändert (zum Beispiel durch Verschiebung eines der Kreise), wirkt sich dies deshalb auf die Position beider Kreise aus.

Nutzen wir die erste Implementierung, wird hingegen ein neues `Point`-Objekt als Position der Kopie angelegt. Dadurch lassen sich jetzt die Positionen der beiden Kreise unabhängig voneinander ändern.

Schließlich wollen wir uns noch die spezielle `this`-Referenz genauer ansehen. Innerhalb einer Objektmethode ist `this` eine Referenz auf das Objekt, für das die Methode aufgerufen wurde. In Klassenmethoden macht `this` dagegen keinen Sinn, da zum Aufruf solcher Methoden ja kein Objekt benötigt wird.

Betrachten wir zum Beispiel den Fall, dass wir mit dem Konstruktor-Aufruf

```
Point p = new Point(2, 3);
```

ein Objekt `p` vom Typ `Point` erstellt haben. Rufen wir im Anschluss die Methode

```
p.setX(5);
```

auf, um die X-Koordinate des Punktes zu verändern, wird dadurch im Rumpf von `setX` die Wertzuweisung

```
this.x = x;
```

ausgeführt. Hier ist `this` also das Objekt, für welches `setX` aufgerufen wurde, in unserem Beispiel `p`. Von diesem Objekt können wir mit dem Punkt-Operator die Attribute abfragen. Mit `this.x` ist also das Attribut `x` von `p` gemeint, dem ein neuer Wert zugewiesen wird. Dieser neue Wert heißt ebenfalls `x` (genau so wie das Attribut). Durch die Verwendung der `this`-Referenz ist aber genau klar, dass mit dem linken Teil der Zuweisung das Attribut von `p` und mit dem rechten Teil der Parameter der Methode `setX` gemeint ist.

Die `this`-Referenz lässt sich übrigens auch zurückgeben. So verändern etwa Methoden der Klasse `StringBuilder` das Objekt und geben mit

```
return this;
```

(eine Referenz auf) das (veränderte) Objekt zurück.

Schließlich kann man innerhalb eines Konstruktors in der ersten Zeile mit

```
this(<Werte>)
```

auf einen anderen Konstruktor zugreifen. Dieser wird automatisch anhand der Anzahl und Typen der übergebenen Werte zugeordnet. So vermeidet man unnötige Redundanz von Code.

Merke 4.28: this-Referenz

- Innerhalb einer Objektmethode kann `this` als eine Referenz auf das Objekt, für das die Methode aufgerufen wurde, verwendet werden.
- Innerhalb eines Konstruktors kann `this` in der ersten Zeile als eine Referenz auf einen anderen Konstruktor verwendet werden.

4.2.6 Vergleich von Java und C

Referenzen in Java entsprechen Zeigern in C. In Java sind damit alle Objekte Zeiger auf einen strukturierten Datentyp. Im Unterschied zu C können wir in Java also nicht auswählen, ob wir einen Zeiger auf einen strukturierten Datentyp oder den strukturierten Datentyp selbst benutzen. Stattdessen übernimmt Java die Verwaltung von Zeigern selbst und geht insbesondere davon aus, dass man jedes Objekt nach dem Call-by-Reference-Prinzip verwenden will. Gibt eine Methode einen Referenz-Datentyp zurück, so wird zudem automatisch ein Zeiger anstelle eines erzeugten Objektes zurückgegeben.

Die Erzeugung von Objekten mittels Konstruktoraufruf entspricht der Verwendung der Funktion `malloc` zur dynamischen Speicherreservierung in C. Eine explizite Freigabe des reservierten Speichers wie mit der Funktion `free` in C (zur Vermeidung sogenannter Speicherlecks) ist in Java nicht notwendig, das erledigt automatisch die Garbage Collection für alle nicht mehr referenzierten Speicherbereiche.

Zudem gibt es in Java keine expliziten Zeiger auf primitive Datentypen. Entsprechend können wir im Unterschied zu C in Java also auch nicht auswählen, ob wir einen Zeiger auf einen primitiven Datentyp oder den primitiven Datentyp selbst benutzen.

Damit ist in Java für alle Datentypen festgelegt, ob Variablen von diesem Typ Zeiger sind oder nicht. Deshalb gibt es in Java auch keinen Operator zur Dereferenzierung eines Zeigers und keinen Adressoperator. Alle Objekte sind adresswertig und deren Dereferenzierung erfolgt automatisch. Die Handhabung von Referenzen in Java ist damit deutlich einfacher als die Handhabung von Zeigern in C.

4.3 Vererbung

Vererbung bezeichnet in objektorientierter Programmierung nicht dasselbe wie das, was wir im normalen Sprachgebrauch unter Vererbung verstehen. Insbesondere bezieht sich Vererbung in objektorientierter Programmierung nicht auf Objekte wie Personen, sondern auf Klassen. Eine Vererbungs-Beziehung besteht zwischen einer sogenannten **Unterklasse** und einer sogenannten **Oberklasse**, wenn alle Objekte der Unterklasse auch zur Oberklasse gehören. Das drücken wir auch so aus: Jedes Objekt der Unterklasse **ist ein** Objekt der Oberklasse. Dafür gibt es viele Beispiele aus dem Alltag:

- Eine *Vorlesung* **ist eine** *Lehrveranstaltung*.
- Ein *Seminar* **ist eine** *Lehrveranstaltung*.
- Eine *Video* **ist eine** *Aufnahme*.
- Ein *Musikstück* **ist eine** *Aufnahme*.
- Eine *Skulptur* **ist ein** *Kunstobjekt*.
- Ein *Bild* **ist ein** *Kunstobjekt*.
- Ein *E-Bike* **ist ein** *Fahrrad*.
- Ein *Fahrrad* **ist ein** *Fahrzeug*.
- Ein *PKW* **ist ein** *Fahrzeug*.
- Ein *Rechteck* **ist eine** *Form*.

- Ein *Kreis* **ist eine** *Form*.
- Ein *mit einer Zahl beschrifteter Kreis* (*Zahlkreis*) **ist ein** *Kreis*.

Objekte der Unterklasse besitzen alle (nicht-privaten) Eigenschaften der Oberklasse – sie **erben** diese Eigenschaften:

- Wollen wir in einer Anwendung **Aufnahmen wie Musikstücke und Videos verwalten**, so wollen wir vielleicht für jede *Aufnahme* ihren *Titel* und ihre *Dauer* speichern. *Titel* und *Dauer* sind Eigenschaften jeder *Aufnahme*. Da ein *Musikstück* eine *Aufnahme* ist, hat es auch diese beiden Eigenschaften *Titel* und *Dauer*. Dasselbe gilt für ein *Video*.
- Wollen wir in einer Anwendung **Kunstobjekte wie Skulpturen und Bilder verwalten**, so benötigen wir (unter anderem) Daten über *Preis*, *Künstler* und *Titel* eines jeden *Kunstobjekts*. Das sind Eigenschaften aller *Kunstobjekte*, und deshalb auch von *Skulpturen* und *Bildern*.
- Wollen wir **mit einem Editor Formen zeichnen**, so verwalten wir zum Beispiel Daten zu *Position* und *Randfarbe* von *Formen*. Diese Eigenschaften haben dann insbesondere auch *Kreise* und *Rechtecke*. *Zahlkreise* wiederum besitzen zusätzlich alle Eigenschaften von *Kreisen*, wie zum Beispiel einen *Radius*.

Zudem definieren Unterklassen zusätzliche Eigenschaften ihrer Objekte. Hat eine Oberklasse mehrere Unterklassen, definieren diese Unterklassen dabei unterschiedliche zusätzliche Eigenschaften:

- Ein *Musikstück* zum Beispiel hat neben *Titel* und *Dauer* die Eigenschaft *Komponist*. *Komponist* ist aber keine Eigenschaft eines *Videos*. Stattdessen haben *Videos* unter anderem die zusätzliche Eigenschaft *Auflösung* (und das ist wiederum keine Eigenschaft eines Musikstücks).
- Während ein *Bild* als zusätzliche Eigenschaften zu denen eines *Kunstobjekts* zum Beispiel *Höhe*, *Breite*, *Untergrund* und *Farbmedium* hat, sind *Durchmesser*, *Höhe*, *Gewicht* und *Material* mögliche zusätzliche Eigenschaften einer *Skulptur*.
- Ein *Kreis* hat gegenüber allgemeinen *Formen* die zusätzliche Eigenschaft *Radius*, ein *Rechteck* hingegen *Höhe* und *Breite*. Ein *Zahlkreis* wiederum hat gegenüber allgemeinen *Kreisen* die zusätzlichen Eigenschaften *Zahl* und *Schriftgröße*.

Die zusätzlichen Attribute unterscheiden die Unterklasse von der Oberklasse und machen die Unterklasse zu einer **Spezialisierung** der Oberklasse. Umgekehrt ist die Oberklasse eine **Verallgemeinerung** der Unterklasse. Dadurch umfasst die Oberklasse mehr Objekte als die Unterklasse.

Manche Oberklassen sind hierbei **abstrakt** – das sind Oberklassen, deren Objekte ausschließlich zu ihren Unterklassen gehören. Beispielsweise bilden *Formen* eine abstrakte Oberklasse der Unterklassen *Kreis* und *Rechteck*, denn *Form* ist lediglich ein abstrakter Oberbegriff für bestimmte Sorten konkreter Formen, die wir zeichnen können. Insbesondere fehlen uns auf dieser abstrakten Ebene die notwendigen Informationen zum Zeichnen des Objekts. Ebenso bilden *Kunstobjekt* und *Aufnahme* abstrakte Oberklassen.

Demgegenüber ist die Klasse *Kreis* eine **nicht abstrakte (konkrete)** Oberklasse von *Zahlkreis*. Denn es gibt Kreise, die nicht mit einer Zahl beschriftet sind. Ebenso ist die Klasse *Fahrrad* eine konkrete Oberklasse von *E-Bike*. *Fahrzeug* wiederum ist abstrakt.

Die Beziehung zwischen Unterklassen und Oberklassen sieht in UML am Beispiel von Formen, Rechtecken, Kreisen und Zahlkreisen wie in Abbildung 5 aus. Die Pfeile führen hierbei **von der Unterklasse zur Oberklasse**. Diese Beziehung bedeutet, dass Objekte der Unterklassen alle nicht-privaten Attribute und Methoden der Oberklasse besitzen. Deshalb werden die Attribute und Methoden der Oberklasse in den Unterklassen grundsätzlich nicht mehr angegeben (Ausnahmen von dieser Regel werden gleich erklärt). Abstrakte Klassen werden über das Schlüsselwort `abstract` gekennzeichnet.

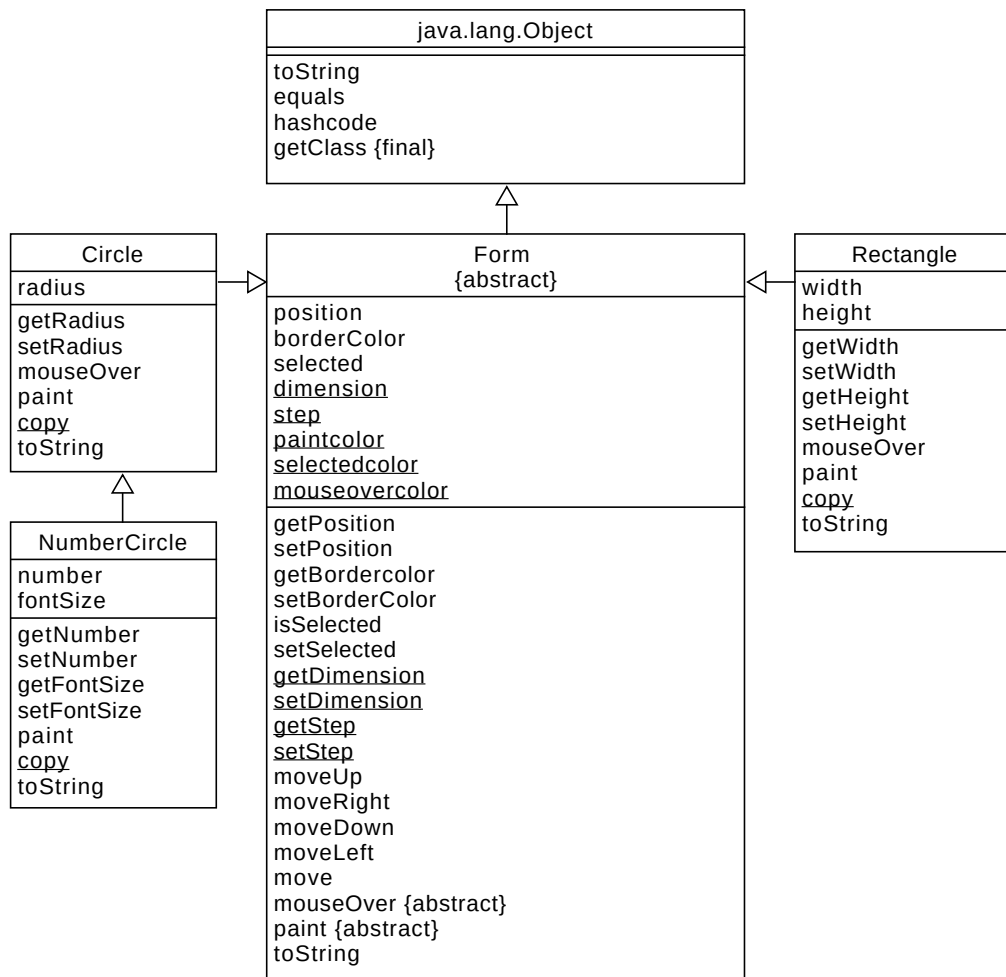


Abbildung 5: Vererbungshierarchie für Formen.

Zusätzlich ist in Abbildung 5 zudem die Java-Wurzelklasse `Object` angegeben. Diese ist Oberklasse aller Klassen in Java. Damit können die Methoden der Klasse `Object` (von denen 4 angegeben sind) in allen Klassen benutzt werden. Von den gezeigten `Object`-Methoden wurde `toString` schon erklärt: Diese Methode definiert die Repräsentation eines Objekts als String und wird zum Beispiel für Ausgaben auf Kommandozeile benutzt. In der Regel möchte man, dass dieser String den Inhalt eines Objekts angibt. Möchte man für eine Unterklasse eine spezifischere Repräsentation des

Inhalts als die der Klasse `Object`, muss man `toString` in der Unterklasse überschreiben. In diesem Fall schreibt man `toString` noch einmal in die Unterklasse. Das ist hier für alle Unterklassen der Fall.

Die weiteren gezeigten `Object`-Methoden besprechen wir später. Der Zusatz `final` bei `getClass` bedeutet, dass `getClass` nicht überschrieben werden kann.

Wir sehen in Abbildung 5, dass alle Formen eine Position, eine Randfarbe und eine einheitliche Anfangsgröße haben und ausgewählt sein können. Zudem haben alle Formen dieselben möglichen Randfarben. In den anderen Eigenschaften unterscheiden sich spezifische Formen wie Kreise und Rechtecke. Zahlkreise haben zusätzlich zu Kreisen eine Zahl und eine Schriftgröße. Methoden zur Verwaltung der Attribute der Oberklasse schreibt man in die Oberklasse. Also befinden sich alle Getter und Setter zu Oberklassenattributen auch in der Oberklasse. Methoden zur Verwaltung der Attribute einer Unterklasse schreibt man entsprechend in die Unterklasse. Da die Attribute der Oberklasse auch Attribute der Unterklasse sind, kann man für jedes Objekt einer Unterklasse alle Methoden der Oberklasse aufrufen – Unterklassen-Objekte erben neben den nicht-privaten Attributen auch die nicht-privaten Methoden der Oberklasse.

Ein besondere Rolle spielen in abstrakten Oberklassen sogenannte abstrakte Methoden, die man am Zusatz `abstract` erkennt. Das sind Methoden, die in der Oberklasse noch nicht implementiert werden können, weil dafür Informationen fehlen. Das trifft zum Beispiel auf die Methode `mouseover` zu. Diese kann nur implementiert werden, wenn die genauen Begrenzungen der Form bekannt sind. Diese genauen Begrenzungen sind aber nur in den konkreten Unterklassen bekannt. In den Unterklassen werden abstrakte Methoden der Oberklasse nochmals angegeben, da sie dort dann konkret implementiert werden. Hierbei unterscheiden sich dann die Implementierungen in den verschiedenen Unterklassen. Mit abstrakten Methoden stellt man sicher, dass jede Unterklasse diese Methode zur Verfügung stellt, ohne sie schon in der Oberklasse konkret definieren zu müssen. Mehr zu diesem Thema und zur Implementierung in Java besprechen wir in den folgenden Unterkapiteln.

Zur Darstellung von Vererbungsbeziehungen ist es übrigens nicht notwendig, Attribute und Methoden anzugeben – es genügen die Klassennamen. Für die Implementierung in Java benötigen wir aber natürlich Attribute und Methoden.

Was ist aber nun der Vorteil dieser Modellierung?

Das sieht man sofort, wenn man sich eine alternative Modellierung von Formen ohne Vererbung in Abbildung 6 ansieht (hier haben wir aus Gründen der Übersichtlichkeit keine Methoden angegeben). In diesem Fall müsste man die gemeinsamen Attribute und Methoden der Unterklassen in jeder Unterklasse separat angeben und implementieren. Das führt zu redundanten Informationen, die den Code schwerer wartbar und erweiterbar machen.

Wie wir an dem Beispiel beobachten können, kann die Unterklasse einer Oberklasse selbst wieder Oberklasse einer anderen Klasse sein. So entstehen sogenannte **Vererbungshierarchien**. Tatsächlich bilden alle Klassen der Java-API eine solche Vererbungshierarchie. In Abbildung 7 sehen wir einen Ausschnitt aus dieser Vererbungshierarchie in einfacher UML-Darstellung. Vollziehen Sie diese Grafik in der Java-API nach. Die Klasse `Object` ist die einzige Klasse ohne Oberklasse. Alle

NumberCircle	Circle	Rectangle
position borderColor selected <u>dimension</u> step paintcolor selectedcolor <u>mouseovercolor</u> radius number fontSize	position borderColor selected <u>dimension</u> step paintcolor selectedcolor <u>mouseovercolor</u> radius	position borderColor selected <u>dimension</u> step paintcolor selectedcolor <u>mouseovercolor</u> width height

Abbildung 6: Eine Modellierung ohne Vererbung.

Klassen sind direkt oder indirekt Unterklassen von Object. Zum Beispiel ist Number eine direkte Unterklasse von Object. Integer wiederum ist eine direkte Unterklasse von Number, und damit eine indirekte Unterklasse von Object. Integer erbt deshalb alle nicht-privaten Methoden der Klassen Number und Object.

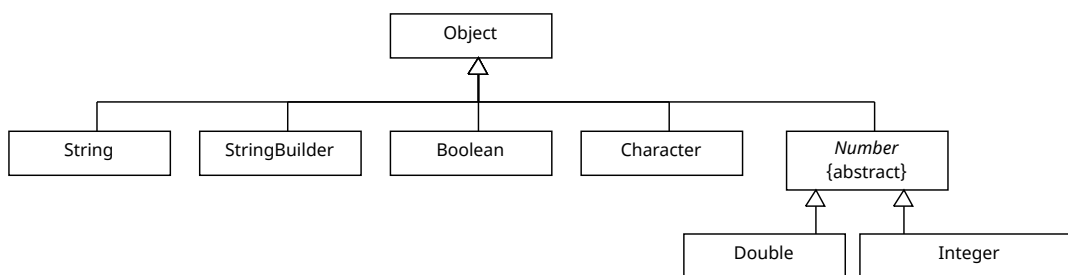


Abbildung 7: Eine Ausschnitt der Vererbungshierarchie der Java-API.

Wir können mit dem Prinzip der Vererbung auch explizit Spezialisierungen von API-Klassen erstellen. Die API bietet zum Beispiel die Klasse `JFrame` für leere Fenster. In einer Anwendung benötigen wir aber für die grafische Benutzerschnittstelle Fenster mit vielen verschiedenen Komponenten wie Schaltflächen, Textfelder, Menüs, und so weiter. Ein solches (anwendungs-)spezifisches Fenster erstellen wir als Unterklasse der API-Klasse `JFrame`. Dadurch stehen uns alle Grundfunktionalitäten der Klasse `JFrame` (wie die zum Schließen des Fensters) auch in unserer speziellen Unter-Fensterklasse zur Verfügung. Allerdings darf man nicht von allen API-Klassen Unterklassen bilden. Einige Klassen verhindern das durch die Kennzeichnung mit dem Schlüsselwort `final`. Dazu zählt zum Beispiel die Klasse `String`.

Definition 4.29: Finale Klassen

Von finalen Klassen können keine Unterklassen gebildet werden.

4.3.1 Die Klasse `Object`

Die Klasse `Object` stellt einige wichtige Methoden für alle Klassen zur Verfügung.

```
public String toString():
```

Der Aufruf `o.toString()` für ein Objekt `o` konvertiert dieses Objekt in einen String und gibt diesen zurück. In jedem Aufruf von `System.out.println(o)` wird diese Methode implizit auf `o` angewendet. Da alle anderen Klassen diese Methode erben, können somit Objekte beliebiger Klassen in einen String konvertiert und so ausgegeben werden. In der Standardimplementierung der Klasse `Object` liefert `toString` einen String bestehend aus dem Klassennamen und der Speicheradresse des Objektes. Das sind genau die Informationen, die allgemein über jedes Objekt, egal welcher Klasse, vorliegen. In der Regel überschreiben Unterklassen diese Standardimplementierung für eine spezifischere Rückgabe des Inhalts ihrer Objekte. Zum Beispiel liefert die Implementierung von `toString` der Klasse `String` den String und in der Einleitung haben wir `toString` unserer Klasse `Person` spezifisch so implementiert, dass die Attributwerte eines `Person`-Objekts zurückgegeben werden. Später diskutieren wir das Thema **Überschreiben von Methoden in Unterklassen** noch genauer.

```
public boolean equals(Object obj):
```

Der Aufruf `o.equals(obj)` für zwei Objekte `o` und `obj` überprüft, ob diese beiden Objekte gleich sind. In der Standardimplementierung der Klasse `Object` wird der Wert `true` zurückgegeben, wenn beide Objekte an derselben Speicheradresse stehen (sie also **identisch** sind), und sonst `false`. Um nicht die Gleichheit der Speicheradresse, sondern die Gleichheit des Inhalts zu testen, muss eine Klasse `equals` entsprechend wieder geeignet überschreiben. Probieren Sie dazu die Implementierung von `equals` der Klasse `String` aus.

```
public int hashCode():
```

Der Aufruf `o.hashCode()` liefert einen sogenannten **Hashcode** für `o`. Dieser kann in verschiedenen anderen Klassen, wie zum Beispiel `HashMap` benutzt werden, um Daten effizienter zu verarbeiten, und wird in der Regel für Unterklassen ebenfalls überschrieben.

```
public final Class<?> getClass():
```

Der Aufruf `o.getClass()` liefert ein `Class`-Objekt, das die Klasse von `o` repräsentiert (in Java werden auch die Klassen selbst wie Objekte verwaltet). Über dieses `Class`-Objekt kann zum Beispiel der Name der Klasse, ihre Attribute, Methoden und Oberklasse abgefragt werden (finden Sie die entsprechenden Methoden und probieren Sie diese aus!). Bei dem `<?>` handelt es sich um einen sogenannten **Wildcard**-Operator. Dieser kann durch eine beliebige Klasse ersetzt werden – in diesem Fall durch die Klasse des Objekts, für das `getClass` aufgerufen wurde. Auch diese Methode können wir also in allen Klassen benutzen. Im Unterschied zu den bisherigen Methoden wird `getClass` aber nicht in Unterklassen überschrieben. Das ist auch gar nicht erlaubt, da `getClass` final ist. Hier steht also nur die Standardimplementierung der Klasse `Object` zur Verfügung. Das kann man sich so vorstellen, dass die Klasse `Object` ein privates Attribut `class` vom Typ `Class<?>` hat, das mit `getClass` abgefragt wird. Hierfür werden keine spezifischen Informationen aus der Unterklasse benötigt.

4.3.2 Oberklassen

Wir betrachten das, was wir bisher von Klassen-Vererbung gesehen haben, nun etwas formaler.

Definition 4.30: Unter- und Oberklassen

Seien U und O jeweils Klassen und U wie folgt deklariert:

```
class U extends O
```

- O nennt man **(direkte) Oberklasse** von U . In Java kann eine Klasse **höchstens eine Oberklasse** haben.
- U nennt man **(direkte) Unterklasse** von O . Zu einer Klasse kann es mehrere verschiedene Unterklassen geben.
- O kann Unterklasse einer anderen Oberklasse sein, und U kann Oberklasse einer anderen Unterklasse sein. Eine Oberklasse von O ist auch eine Oberklasse von U . Eine Unterklasse von U ist auch eine Unterklasse von O . Die Beziehung zwischen Unterklassen und Oberklassen ist dabei **hierarchisch**, darf also keine Zyklen bilden. So entstehen sogenannte **Vererbungshierarchien**.
- Die Klasse `Object` ist automatisch **Oberklasse aller Klassen**, ohne dass man dies explizit deklarieren muss.
- Von **finalen** Klassen dürfen keine Unterklassen gebildet werden.
- In anderen Programmiersprachen (z.B. C++) kann eine Klasse auch mehrere Oberklassen haben (das bezeichnet man als **Mehrfachvererbung**)

In unserem vorangegangenen Beispiel sind also `Circle` und `Rectangle` jeweils Unterklassen von `Form` und `Form` jeweils eine Oberklasse von `Circle` und `Rectangle`. In der API sind zum Beispiel `Integer` und `Double` jeweils Unterklassen von `Number`. `Integer`, `Double` und `Number` sind jeweils Unterklassen von `Object`. Die Klassen `Integer` und `Double` sind dabei final.

Definition 4.31: Vererbung

Sei U eine Unterklasse von O .

- Alle nicht-privaten Attribute von O sind auch Attribute von U . Man sagt, O **vererbt seine Attribute** an U . U kann zusätzliche Attribute haben. Nur noch diese werden im Rumpf von U definiert.
- Alle nicht-privaten Methoden von O sind auch Methoden von U . Man sagt, O **vererbt seine Methoden** an U . U kann zusätzliche Methoden haben. Nur noch diese werden im Rumpf von U definiert.
- Konstruktoren werden **nicht** vererbt!
- Jedes U -Objekt ist ein O -Objekt.
- U ist eine **Spezialisierung** von O . O ist eine **Generalisierung** von U .

Mit diesen Begriffen sind also `Circle` und `Rectangle` jeweils Spezialisierungen von `Form` und `Form` ist eine Generalisierung von `Circle` und `Rectangle`. In den Klassen `Form`, `Circle` und `Rectangle` stehen alle Methoden der API-Klasse `Object` zur Verfügung, und in den Klassen `Circle` und `Rectangle` alle Methoden der Klasse `Form`.

Abbildung 8 zeigt, wie man Ober- und Unterklassen in UML angibt.

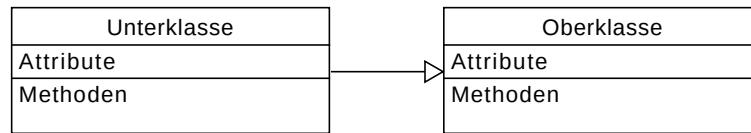


Abbildung 8: Angabe von Ober- und Unterklassen in UML.

4.3.3 Substitutionsprinzip

Da eine Unterklasse über alle Attribute und Methoden ihrer Oberklasse verfügt, kann eine Variable vom Typ einer Oberklasse jederzeit den Inhalt eines Unterklassen-Objekts referenzieren. Zum Beispiel kann man schreiben:

```
Object o = new String("Hello");
```

Hier referenziert die Variable `o` vom Oberklassen-Typ `Object` den Inhalt eines Objekts vom Unterklassen-Typ `String`. In diesem Fall stehen für `o` aber **nur** die Methoden der Oberklasse zur Verfügung, denn der Compiler überprüft bei der Übersetzung des Quellcodes, ob eine für `o` aufgerufene Methode für den angegebenen Typ von `o` verfügbar ist. Damit kann man zum Beispiel

```
o.toString()
```

aufrufen, denn `toString` ist eine Methode des Typs `Object` von `o`. Andererseits kann man **nicht**

```
o.length()
```

aufrufen, denn `length` ist **keine** Methode der Klasse `Object`.

Inwiefern macht das aber Sinn, wo wir uns dadurch doch bei der Benutzung von Methoden selbst einschränken?

Betrachten wir dazu unser eigenes Beispiel eines grafischen Editors zum Zeichnen von Kreisen und Rechtecken. Hier wollen wir zum Beispiel in der Anwendung eine Menge verschiedener gezeichneter Kreise und Rechtecke markieren und gemeinsam verschieben. Dazu können wir wie folgt vorgehen. Zuerst sammeln wir alle markierten Kreise und Rechtecke in einem Array vom Oberklassen-Typ `Form`:

```
Form[] auswahl;
```

Dann durchlaufen wir alle Kreise und Rechtecke im Array in einer Schleife und setzen jeweils die Position neu. Die dafür benötigte Methode gehört zur Oberklasse `Form`, denn `position` ist ein Oberklassen-Attribut. Ohne Benutzung der Oberklasse hätten wir für Kreise und Rechtecke jeweils ein eigenes Array anlegen müssen. Dadurch wäre der Code weniger allgemein und redundant gewesen. Auf der anderen Seite hat uns die Einschränkung bei der Verfügbarkeit der Methoden gar nicht beeinträchtigt, denn für die beschriebene Anwendung waren nur Oberklassen-Methoden nötig.

Dasselbe gilt auch für die Übergabe von Argumenten an Methoden. Betrachten wir dazu die folgende Methode der Klasse `Object`:

```
boolean equals(Object obj)
```

Der Eingabeparameter `obj` ist vom Oberklassen-Typ `Object`. Für diesen können wir bei Aufruf der Methode beliebige Unterklassen-Objekte als Argument übergeben:

```
Object o = new Object();  
String s = new String();  
boolean b = o.equals(s);
```

Hier übergeben wir ein Objekt `s` des Unterklassen-Typs `String` für `obj`. Damit können wir `o` und `s` (und allgemein beliebige Objekte auch unterschiedlichen Typs) auf Gleichheit überprüfen.

Merke 4.32: Substitution

- Ein Objekt `o` eines Oberklassen-Typs kann **Inhalte verschiedenen Typs haben**, denn der Inhalt kann jeden Unterklassen-Typ des Typs von `o` haben.
- Für den Aufruf von Methoden für `o` ist **der Typ von `o` ausschlaggebend**, **nicht** der Typ des referenzierten Inhalts.
- Dadurch wird Code **allgemeiner und für mehr Objektarten benutzbar** und damit **weniger redundant**.

Definition 4.33: Substitutionsprinzip

Sei U eine Unterklasse von O .

- Man kann einem Objekt o vom Typ O eine U -Referenz als Wert zuweisen.

`O o = <U-Referenz>;`

- Man kann für einen Eingabeparameter o vom Typ O eine U -Referenz als Argument übergeben. Eine Methode mit einer Deklaration der Form

`<Sichtbarkeit> [abstract] [final] [static]
<Typ> <Methode> (O o, <weitere Parameter>)`

kann man also wie folgt aufrufen:

`<Methode>(<U-Referenz>, <weitere Argumente>);`

- Hierbei ist `<U-Referenz>` jeweils eine Referenz auf den Inhalt eines Objekts vom Typ U .
- Für o können jeweils nur die Methoden der Klasse O aufgerufen werden, aber nicht die Methoden der Klasse U .

Umgekehrt klappt das nicht! Die Anweisung

```
String s = new Object();
```

führt zu einem Compilerfehler, denn der Inhalt vom Typ `Object` (rechte Seite) ist nicht spezifisch genug für den Typ `String` von `s` (linke Seite). Deshalb können für diesen Inhalt auch nicht sinnvoll `String`-Methoden aufgerufen werden.

Betrachten wir noch einmal ein Objekt o vom Typ O mit dem Inhalt vom Typ einer Unterklasse U . Für o stehen dann also **nur** die Methoden von O zur Verfügung. Manchmal wollen wir aber trotzdem eine Unterklassen-Methode für o aufrufen. Sind wir uns sicher, dass der Inhalt von o eigentlich vom Typ U ist, so können wir o auf U **casten**. Das sieht in unserem Eingangsbeispiel wie folgt aus:

```
Object o = new String("Hello");  
String s = (String) o;  
int n = s.length();
```

Durch `(String)o` wird der Typ `Object` von o in den Unterklassen-Typ `String` umgewandelt. Diese Operation nennt man **Typcast**. Dieser funktioniert nur, falls der Inhalt eines Objekts tatsächlich den Unterklassen-Typ aus dem Typcast hat. Nach dem Typcast stehen alle Methoden der Klasse `String` zur Verfügung. Entsprechend können wir nun, im Gegensatz zu vorher, die Methode `length` aufrufen. Im obigen Beispiel haben wir dazu das Objekt mit dem neuen Typ noch durch die Wertzuweisung in `s` umbenannt. Das ist aber nicht unbedingt notwendig. Der Code würde auch wie folgt funktionieren:

```
Object o = new String("Hello");  
int n = ((String) o).length();
```

Definition 4.34: Typcast

Sei U eine Unterklasse von O . Sei o ein Objekt vom Typ O mit dem Inhalt vom Typ U . Dann lässt sich o wie folgt in ein Objekt u vom Typ U mit demselben Inhalt **casten**.

$$U \ u = (U) \ o;$$

Für u stehen dann **alle** Methoden der Klasse U zur Verfügung.

Ist U **keine** Unterklasse von O , führt ein solcher Typcast zu einem Laufzeitfehler.

4.3.4 Polymorphismus

Wie wir in Abschnitt 4.3.1 gesehen haben, sind die von der Basisklasse `Object` angebotenen Implementierungen von Methoden wie etwa `toString` (oder auch `equals`) nicht unbedingt das, was wir uns erhoffen. Insbesondere wenn wir ein Objekt auf der Kommandozeile ausgeben wollen (und es entsprechend an die Methode `System.out.println` übergeben), erhoffen wir uns wohl in den seltensten Fällen die Ausgabe des Klassennamens und der Speicherstelle, an welcher das Objekt gespeichert ist. Es kann entsprechend sehr schnell der Wunsch entstehen, eine andere `toString`-Methode zur Verfügung zu haben, mit der alle definierenden Eigenschaften unseres konkreten Objektes schön lesbar auf der Kommandozeile ausgegeben werden können.

Und tatsächlich ist dies möglich – solange eine Methode der Oberklasse nicht als `final` deklariert wurde, können wir diese in der Unterklasse neu definieren, als würde sie nicht bereits von der Oberklasse zur Verfügung gestellt werden. Wir sagen dann, dass wir diese Methode der Oberklasse **überschreiben**.

Zum Beispiel überschreibt die Klasse `String` die Methode `toString` der Klasse `Object`. Während im folgenden Code

```
Object o = new Object();  
o.toString() // gibt die Speicheradresse von o aus
```

die Speicheradresse von `o` ausgegeben wird, wird durch den Code

```
String s = new String("Hello");  
s.toString() // gibt Hello aus
```

die verwaltete Zeichenfolge `"Hello"` ausgegeben. Ebenso überschreibt unsere Klasse `Person` aus dem einleitenden Beispiel die Methode `toString` der Klasse `Object`.

Definition 4.35: Methoden in Unterklassen überschreiben

Sei U eine Unterklasse von O . Dann kann U eine nicht private, nicht finale Methode von O durch eine Methode mit gleichem Methodenkopf und anderer Implementierung überschreiben.

In zwei speziellen Fällen darf auch der Methodenkopf abweichen:

- Erweiterung der Sichtbarkeit der Methode in U .
- Spezialisierung des Rückgabetyps der Methode in U (man spricht dann von **kovarianten Rückgabewerten**).

Bei der Modellierung / Implementierung gibt es folgende Regeln:

- In Java stellen wir in U die Annotation `@Override` voran.
- In UML werden überschriebene Methoden auch in U angegeben.

Derselbe Methodenaufruf für Objekte verschiedener Unterklassen einer Oberklasse führt dann zur Ausführung unterschiedlicher Implementierungen.

Wir wissen nun also das folgende:

- Eine Methode kann für ein Objekt o aufgerufen werden, wenn die Methode zum Typ von o oder zum Typ einer direkten oder indirekten Oberklasse von o gehört.
- Der Inhalt, den ein Objekt o referenziert, kann einen anderen Typ haben als o selbst, nämlich den einer direkten oder indirekten Unterklasse von o .
- In diesem Unterklassen-Typ kann die Methode überschrieben sein, das heißt eine andere Implementierung haben als für den Typ von o .

Aber welche Implementierung wird dann beim Aufruf der Methode auf o ausgeführt?

Die für den Oberklassen-Typ von o oder die für den Unterklassen-Typ des Inhalts von o ? Eine Antwort auf diese durchaus berechtigte Frage liefert die folgende Definition:

Definition 4.36: Polymorphismus

Beim Übersetzen des Programms muss für ein Objekt (noch) nicht bekannt sein, zu welchem (Unterklassen-)Typ sein Inhalt gehört. Zur Laufzeit wird dann beim Aufruf die Implementierung für den Typ des Inhalts benutzt. Man spricht hierbei vom **späten Binden** (der Methode an den Inhalt des Objekts).

Wandeln wir unser Beispiel also ein wenig ab, sodass wir den folgenden Code erhalten:

```
Object o = new String("Hello");
o.toString() // gibt Hello aus
```

Das Objekt o hat also den Typ `Object` und referenziert Inhalt vom Typ `String`. Führen wir diesen Code aus, so wird auf Kommandozeile `Hello` ausgegeben, obwohl `toString()` der Klasse `Object` nur den Klassennamen und die Speicherstelle ausgibt. Dies ist dem *späten Binden* zu verdanken, gemäß

dem automatisch geprüft wird, welche konkrete Implementierung der überschriebenen Methode ausgeführt werden soll.

Ruft man also eine Oberklassen-Methode auf, die in Unterklassen überschrieben wird, wird erst bei der Ausführung die zum Inhalt des Objekts passende (spezialisierte) Unterklassenimplementierung ausgewählt und ausgeführt.

Beispiel 4.37: Implementierung der Methode `println(Object o)`

Diese `println`-Variante ist wie folgt implementiert:

```
public void println(Object o) {
    String s = String.valueOf(o);
    print(s);
    println();
}
```

Die `valueOf`-Methode wiederum ruft `o.toString` für das übergebene Objekt `o` auf, was möglich ist, da `toString` in der Klasse `Object` zur Verfügung steht:

```
public static String valueOf(Object obj) {
    return obj.toString();
}
```

Wird zur Laufzeit ein `String`-Objekt für `o` übergeben (Substitutionsprinzip), wird die `toString`-Implementierung der Klasse `String` benutzt (Polymorphismus).

4.3.5 Typ- und Klassenzugehörigkeit

Durch das Substitutionsprinzip entspricht der Typ einer Referenzvariable `o` nicht unbedingt dem Typ des Inhalts. Insbesondere für Typcasts sollten wir aber eine Möglichkeit haben zu überprüfen, von welchem Typ der Inhalt von `o` ist. Hierfür kann man den `instanceOf`-Operator und die bereits eingeführte `instanceOf`-Methode benutzen. Für die Objekte

```
Object s = new String("Hello");
Object o = new Object();
```

bekommt man folgende Ergebnisse:

Ausdruck	Wert	Ausdruck	Wert
<code>s instanceof String</code>	true	<code>s.getClass().equals(String.class)</code>	true
<code>s instanceof Object</code>	true	<code>s.getClass().equals(Object.class)</code>	false
<code>o instanceof String</code>	false	<code>o.getClass().equals(String.class)</code>	false
<code>o instanceof Object</code>	true	<code>o.getClass().equals(Object.class)</code>	true

Mit dem `instanceOf`-Operator wird also getestet, ob der Inhalt eines Objekts den Typ der abgefragten Klasse oder einer Unterklasse dieser Klasse hat (linke Seite der Tabelle). Mit der `getClass`-Methode liefert hingegen vergleichen wir jeweils zwei `Class`-Objekte mit `equals` auf Gleichheit.

Das `Class`-Objekt zu einer Klasse `K` erhalten wir dabei über den Ausdruck `K.class`. Die Methode `getClass` liefert offenbar genau den Typ des Inhalts eines Objekts (rechte Seite der Tabelle).

Definition 4.38: Der `instanceOf`-Operator (Typzugehörigkeit)

Der `instanceOf`-Operator testet die **Typzugehörigkeit in einer Vererbungshierarchie**: Ist `o` ein Objekt und `K` eine Klasse, so liefert der Ausdruck

```
o instanceof K
```

den Wert `true`, falls `o` ein `K`-Objekt ist, das heißt wenn `K` oder eine Unterklasse von `K` der Typ des Inhalts von `o` ist. Ansonsten liefert der Ausdruck `false`.

Achtung: Der Operator `instanceOf` erzeugt einen Programmabbruch, wenn die getesteten Typen keine hierarchische Vererbungsbeziehung haben!

Definition 4.39: Die `getClass`-Methode (Klassenzugehörigkeit)

Die `getClass`-Methode liefert die **Klassenzugehörigkeit**: Ist `o` ein Objekt und `K` eine Klasse, liefert

```
o.getClass().equals(K.class)
```

den Wert `true`, falls `K` genau der Typ des Inhalts von `o` ist, sonst `false`.

Weitere Beispiele zu den Themen Substitution und Polymorphismus finden Sie in *Screencast 06: Die vielgestaltige Welt des Polymorphismus*.

4.3.6 Abstrakte Klassen

In der Einleitung zu diesem Unterkapitel haben wir bereits gesehen, dass es manchmal sinnvoll ist, eine Oberklasse als abstrakt zu deklarieren. Das ist immer dann der Fall, wenn man keine Objekte vom Typ dieser Klasse selbst angelegen möchte, weil alle Objekte zu konkreten Unterklassen gehören. Beispiele, wo dies der Fall ist, sind etwa:

- Formen bilden eine abstrakte Oberklasse der Unterklassen Kreis und Rechteck, denn eine Form ist lediglich ein abstrakter Oberbegriff für eine bestimmte Sorte konkreter Formen, die wir zeichnen können. Ohne Spezialisierung, welche Art einer Form wir meinen, können wir uns unter einem Objekt dieser Klasse nichts vorstellen. Insbesondere fehlen uns auf dieser abstrakten Ebene die notwendigen Informationen über dessen grafische Darstellung zum Zeichnen des Objekts. Bei den Unterklassen Kreis und Rechteck hingegen haben wir diese Informationen.
- Kunstobjekte bilden eine abstrakte Oberklasse für die Unterklassen Bild und Skulptur, denn Kunstobjekt ist nur ein abstrakter Oberbegriff für verschiedene konkrete Arten von Kunstobjekten, die man in einer Anwendung verwalten möchte. Jedes verwaltete Objekt gehört hier zu einer konkreten Unterklasse.
- Ein Beispiel für eine abstrakte API-Klasse ist

```
public abstract class Number extends Object implements
    Serializable
```

Diese Klasse repräsentiert Objekte, die einen numerischen Wert als Inhalt haben und stellt gemeinsame Eigenschaften und Methoden unter anderem für die folgenden konkreten Unterklassen bereit: `Double`, `Float`, `Integer`, `Long`, `Short`.

Abstrakte Klassen stellen also so etwas wie eine **Vorlage oder Schablone für weitere Klassen** dar. Es können **keine Objekte von abstrakten Klassen** erstellt werden, man kann lediglich andere Klassen von ihnen erben lassen.

Definition 4.40: Abstrakte Klasse

Mit dem Schlüsselwort `abstract` werden **abstrakte Klassen** deklariert. Abstrakte Klassen sind Klassen, von denen keine Objekte erzeugt werden können. Im Gegensatz zu nicht-abstrakten Klassen können sie sogenannte **abstrakte Methoden** haben.

- Abstrakte Klassen stellen **gemeinsame Attribute und Methoden** für ihre Unterklassen zur Verfügung.
- Abstrakte Klassen sind **nicht final**, da sie sonst keine Unterklassen besitzen könnten.
- Abstrakte Klassen können Konstruktoren haben. Diese können **nur in Unterklassen aufgerufen** werden.
- Nicht abstrakte Klassen heißen auch **konkret**.

Mit abstrakten Methoden stellen wir sicher, dass alle Unterklassen diese Methoden zur Verfügung stellen, ohne sie schon in der Oberklasse konkret definieren zu müssen.

Definition 4.41: Abstrakte Methode

Mit dem Schlüsselwort `abstract` werden **abstrakte Methoden** deklariert, die keinen Rumpf haben.

- Eine abstrakte Methode stellt einen **gemeinsamen Methodenkopf** für die Unterklassen der abstrakten Oberklasse zur Verfügung.
- Eine nicht-abstrakte Methode heißt **konkret**.
- Eine abstrakte Methode muss von den konkreten Unterklassen der abstrakten Oberklasse durch eine **konkrete** (also nicht abstrakte) Methode überschrieben und implementiert werden.
- Abstrakte Methoden sind **nicht final**, da sie sonst nicht von den Unterklassen überschrieben werden könnten.

Mehrere Sachen erscheinen Ihnen eventuell auf den ersten Blick noch etwas verwirrend:

- **Wieso benötigt eine abstrakte Klasse Konstruktoren?**
- **Was genau ist nun der praktische Vorteil von abstrakten Methoden?**

Diese (und weitere) Fragen klären wir im nächsten Abschnitt anhand eines konkreten Beispiels. Und zwar sehen wir uns dazu einen Ausschnitt der Implementierung der Klassen `Form`, `Circle` und `Rectangle` in Java an.

Weitere Beispiele für abstrakte und nicht-abstrakte Klassen in Java finden Sie in *Screencast 04: Von Abstrakt bis Konkret – Vererbung in Java*.

4.3.7 Fallstudie: Grafischer Editor

Die Implementierung des UML-Diagramms zum grafischen Editor aus der Einleitung zu diesem Unterkapitel (Abbildung 5) sieht in Ausschnitten wie folgt aus:

```
public abstract class Form {
    private Point position;
    private Color bordercolor = Color.black;
    private boolean selected = false;
    private static int dimension = 10;
    private static int step = 5;
    protected static final Color paintcolor = Color.BLACK;
    protected static final Color selectedcolor = Color.BLUE;
    protected static final Color mouseovercolor = Color.RED;

    protected Form(Point position) {
        setPosition(position);
    }
    ...

    public Point getPosition() {
        return new Point(position);
    }

    public void setPosition(Point position) {
        this.position = position;
    }
    ...

    public abstract boolean mouseOver(Point mousePosition);
    ...

    @Override
    public String toString() {
        return this.getClass().getName()
            + "\nx-Position: " + this.getPosition().getX()
            + "\ny-Position: " + this.getPosition().getY();
    }
}

public class Circle extends Form {
    private int radius;

    public Circle(int radius, Point position) {
```

```

        super(position);
        setRadius(radius);
    }
    ...

    public boolean mouseOver(Point mousePosition) {
        int dx = (position.getX() - mousePosition.getX()) * (position
            .getX() - mousePos.getY());
        int dy = (position.getY() - mousePos.getY()) * (position.getY
            () - mousePos.getX());
        if (dx + dy <= radius * radius) {
            setBordercolor(mouseovercolor);
            return true;
        } else {
            setBordercolor(paintcolor);
            return false;
        }
    }
    ...

    public String toString() {
        return super.toString()
            + "\nRadius: " + this.getRadius();
    }
}

public class Rectangle extends Form {
    private int width;
    private int height;

    public Rectangle(int width, int height, Point position) {
        super(position);
        setWidth(width);
        setHeight(height),
    }
    ...

    public boolean mouseOver(Point mousePosition) {
        int dx = Math.abs(position.getX() - mousePosition.getX());
        int dy = Math.abs(position.getY() - mousePos.getY());
        if ((dx <= width) && (dy <= length)) {
            setBordercolor(mouseovercolor);
            return true;
        } else {
            setBordercolor(paintcolor);
            return false;
        }
    }
    ...

    public String toString() {
        return super.toString()

```

```

        + "\nWidth: " + this.getWidth()
        + "\nLength: " + this.getLength();
    }
}

```

Durch das Schlüsselwort `abstract` ist die Klasse `Form` abstrakt und durch das Schlüsselwort `extends` werden die Klassen `Circle` und `Rectangle` zu deren Unterklassen.

Konstruktoren der Oberklasse `Form` sind dafür da, Attribute der Oberklasse zu initialisieren. In diesem Beispiel wird bei Aufruf die Position übergeben. Durch die Sichtbarkeit `protected` können wir den Konstruktor nur in Unterklassen aufrufen. Den Aufruf des Konstruktors in den Unterklassen erfolgt dann mit `super`. Dieser Aufruf muss dabei die **erste Anweisung** sein. Ohne Oberklassen-Konstruktor müssten wir dessen Code in allen Unterklassen wiederholen. Dadurch wäre der Code unübersichtlicher und redundant. Wir lagern also gemeinsame Initialisierungen der Unterklassen in die Oberklasse aus.

Zudem sieht man, dass die Getter und Setter in der Oberklasse wie üblich implementiert werden. Abstrakte Methoden schließen wir in der Oberklasse statt mit einem Methodenrumpf einfach mit einem Strichpunkt ab. In den Unterklassen unterscheiden sich die Implementierungen. Während `mouseOver` in der Klasse `Circle` testet, ob die Mausposition nicht mehr als den Radius vom Kreismittelpunkt entfernt ist, wird in der Klasse `Rectangle` getestet, ob die Mausposition in horizontaler Richtung nicht mehr als die Hälfte der Breite vom Rechtecksmittelpunkt abweicht und in vertikaler Richtung nicht mehr als die Hälfte der Höhe.

Aus den verschiedenen Implementierungen der Methode `toString` lernen wir das folgende:

- Mit `getClass` können wir auf Klasseninformationen zugreifen, zum Beispiel über `getName` auf den Namen des Klassenobjekts, das `getClass` liefert.
- Mit `super.toString()` können wir auf die `toString`-Implementierung der Oberklasse zugreifen und sie in der Unterklasse wiederbenutzen.

Durch die unterschiedliche Implementierung von `toString` in den einzelnen Unterklassen sorgen wir dafür, dass Objekte dieser Klassen jeweils eine spezifische Kommandozeilenausgabe ihres Inhalts ermöglichen.

Durch die Verwendung abstrakter Oberklassen-Methoden (wie `mouseOver`) und unterschiedliche Implementierungen von Methoden in Unterklassen (wie `mouseOver` und `toString`) können wir das Substitutionsprinzip und Polymorphismus benutzen. Zuerst können wir beispielsweise alle Kreise und Rechtecke in einem Array vom Oberklassen-Typ `Form` sammeln (Substitutionsprinzip):

```

Form[] allForms;
// Array initialisieren / Formen hinzufuegen: siehe Unterkapitel 4.5

```

Dann können wir alle Elemente des Arrays in einer Schleife durchlaufen. Jedes Element ist dabei ein Objekt vom Typ `Form` und hat entweder einen Inhalt vom Typ `Circle` oder `Rectangle`. Für all diese Objekte können wir nun beispielsweise durch einen Aufruf von `mouseOver` testen, über welchen Objekten sich eine Mausposition `mousePosition` bei einem Mausklick befindet und diese Objekte dann auswählen:

```

for (int i = 0; i < allForms.length; ++i) {
    if (allForms[i].mouseover(mousePosition))
        allForms[i].setSelected(true);
}

```

Die Methoden `mouseover` und `setSelected` stehen hierbei für die Elemente des Arrays nur deshalb zur Verfügung, weil sie in der Oberklasse angegeben sind. Bei Aufruf von `mouseover` wird dann die passende konkrete Implementierung in den jeweiligen Unterklassen benutzt (Polymorphismus). Ohne Benutzung einer abstrakten Methode in der Oberklasse müssten wir für Kreise und Rechtecke jeweils ein eigenes Array anlegen. Dadurch wäre der Code weniger allgemein und redundant.

Wir stellen mit abstrakten Methoden also sicher, dass eine Unterklasse diese Methoden zur Verfügung stellt, ohne sie schon in der Oberklasse konkret definieren zu müssen. Später werden wir bei der Implementierung des Editors zum Beispiel noch die Methode `paint` besprechen, die sich um das Zeichnen des Objekts kümmert. Auch diese ist dann natürlich in der Oberklasse abstrakt, denn in der Oberklasse haben wir keine Information über die Form des Objekts. In Unterklassen können dann wieder unterschiedliche konkrete Implementierungen der Methode `paint` zur Verfügung stellen.

Auf dieselbe Art und Weise können wir `toString` übergreifend für alle Objekte in einem `Form`-Array benutzen und mit Polymorphismus passende Ausgaben für die Unterklassen erhalten.

Übung: Implementieren Sie die Lücken der Klassen `Circle`, `Rectangle` und `Form`, sowie die komplette Klasse `NumberCircle` und testen Sie das Substitutionsprinzip und Polymorphismus mit `toString` in einer Programmklasse.

4.3.8 Die super-Referenz

Das Schlüsselwort `super` kann man in einer Vererbungshierarchie auf zwei Arten verwenden:

1. Verwendung als Objektreferenz

Innerhalb einer Objektmethode ist `super` eine Referenz auf das Objekt, für das die Methode aufgerufen wurde, auf der Ebene der Oberklasse. Damit kann man in einer überschriebenen Methode auf die Implementierung der Oberklasse zugreifen.

2. Verwendung als Konstruktorreferenz

Innerhalb eines Konstruktors einer Unterklasse kann man in der ersten Zeile mit

```
super(<Werte>)
```

auf einen Konstruktor der Oberklasse zugreifen. Dieser wird automatisch anhand der Anzahl und Typen der übergebenen Werte zugeordnet. So vermeidet man unnötige Redundanz von Code.

4.4 Schnittstellen (Interfaces)

Wie wir schon erkannt haben, ist es in Java nicht möglich, eine Unterklasse von mehreren Oberklassen erben zu lassen. Bisher sind abstrakte Oberklassen mit abstrakten Methoden allerdings die einzige Möglichkeit, die wir kennen, um zu erzwingen, dass eine Klasse bestimmte Methoden anbietet. Daher können wir uns die Frage stellen, wie wir am besten vorgehen, wenn wir einer Unterklasse vorgeben möchten, weitere Methoden anzubieten als nur die, die in der Oberklasse definiert wurden.

Müssen wir alle solche Methoden ebenfalls als abstrakte Methoden in der Oberklasse deklarieren? Bloß nicht! Denn nur, weil eine Unterklasse eine solche Methode anbieten muss, bedeutet das nicht, dass diese Methode auch in anderen Unterklassen überhaupt Sinn ergibt.

Bei solchen Problemen können uns **Schnittstellen** helfen. Sie verhalten sich ähnlich wie abstrakte Klassen, indem sie eine Vorlage liefern, welche Methoden eine Klasse bereitstellen soll. Während eine Klasse aber nur **von einer (abstrakten) Klasse erben kann**, kann sie **beliebig viele Schnittstellen implementieren**. Schnittstellen machen besonders dort Sinn, wo man bestimmte Funktionalitäten unabhängig von der Vererbungshierarchie standardisiert nach außen zur Verfügung stellen möchte.

Ein Beispiel für eine API-Schnittstelle ist

```
public interface CharSequence
```

Diese repräsentiert lesbare Folgen von Zeichen und gibt Methoden für den Zugriff auf solche Zeichenfolgen vor, wie zum Beispiel die folgenden uns bereits bekannten Methoden:

- `char charAt(int index)`: abstrakter Methodenkopf einer Methode, die das Zeichen mit dem angegebenen Index in der Zeichenfolge liefert.
- `int length()`: abstrakter Methodenkopf einer Methode, die die Länge der Zeichenfolge liefert.

Diese Schnittstelle wird zum Beispiel von der uns bereits bekannten Klasse `String` implementiert, aber auch von der Klasse `CharBuffer` für die Ein- und Ausgabe mit Textdateien. Diese Klassen sehen damit für alle durch die Schnittstelle vorgegebenen Zugriffsmethoden eine konkrete Implementierung vor.

Ein weiteres Beispiel ist

```
public interface Comparable<T>
```

Diese Schnittstelle repräsentiert vergleichbare Objekte, für die man eine totale Ordnung definieren kann, wie zum Beispiel Zahlen (numerische Ordnung), Strings (alphabetische Ordnung) und Datumsangaben. Es handelt sich hierbei um eine **parametrisierte Schnittstelle**: Für den Parameter `T` kann ein beliebiger Datentyp eingesetzt werden – dadurch werden dann nur Objekte von diesem Typ verglichen. Entsprechend wird eine Methode für den Vergleich von Objekten vorgegeben:

```
int compareTo(T o)
```

Ein Aufruf der Form `o1.compareTo(o2)` muss einen negativen Wert zurückgeben, wenn `o1` in der Ordnung kleiner als `o2` ist, den Wert 0 zurückgeben, wenn `o1` und `o2` in der Ordnung gleich sind, und einen positiven Wert zurückgeben, wenn `o1` in der Ordnung größer als `o2` ist.

Diese Schnittstelle wird zum Beispiel implementiert von `String`, aber auch von den Klassen `Integer` für die Repräsentation von ganzen Zahlen und `LocalDate` für die Repräsentation von Datumsangaben. Diese Klassen implementieren also jeweils konkret (aber natürlich unterschiedlich) die vorgebene Vergleichsmethode.

Das UML-Diagramm in Abbildung 9 zeigt einen entsprechenden Ausschnitt der API. Schnittstellen sind durch das Schlüsselwort `interface` kenntlich gemacht. In der Abbildung ist jeweils nur der Name der Schnittstellen angegeben. Gleich sehen wir auch ein Beispiel für eine Schnittstelle, in dem auch deren Methoden angegeben sind. Die gestrichelten Pfeile zeigen die Implementierungsbeziehungen zwischen Klassen und Schnittstellen. Die Parametrisierung von Schnittstellen (und Klassen – Beispiele dazu später) wird durch eine gestricheltes Rechteck an der rechten oberen Ecke dargestellt. Wie man sieht, gibt es auch Vererbungsbeziehungen zwischen Schnittstellen (mit der gleichen Bedeutung wie bei Klassen: eine Unterschnittstelle erbt alle Methoden einer Oberschnittstelle).

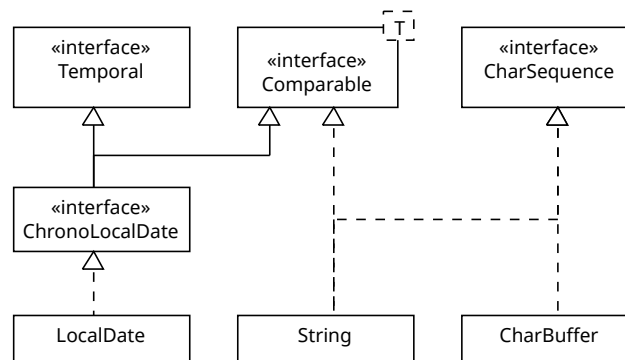


Abbildung 9: Ausschnitt der Java-API zur Implementierung von Schnittstellen.

Die Implementierung von Schnittstellen gibt also eine von der Vererbungshierarchie unabhängige einheitliche Funktionalität von Klassen vor. Wir können API-Schnittstellen auch für eigene Klassen nutzen. Wollen wir zum Beispiel Form-Objekte bezüglich ihres Abstands zum Punkt $(0, 0)$ vergleichen können, dann implementieren wir die Schnittstelle `Comparable<Form>`. Hierfür ersetzen wir den Parameter `T` durch unseren Datentyp `Form`. Abbildung 10 zeigt das zugehörige UML-Diagramm. In diesem sind diesmal die Methoden angegeben. Die Schnittstelle `Comparable<Form>` gibt die Methode `compareTo` vor. Diese schreiben wir dann auch in jede Klasse, die diese Methode implementiert, das ist in diesem Fall die Klasse `Form`. Damit steht `compareTo` wegen der Vererbung auch in allen Unterklassen von `Form` zur Verfügung.

Wir wollen `compareTo` in `Form` so implementieren, dass damit Objekte bezüglich ihres sogenannten Manhattan-Abstands vom Punkt $(0, 0)$ verglichen werden. Der Manhattan-Abstand ist hierbei definiert als die Summe des Abstands von $(0, 0)$ in x-Richtung und des Abstands von $(0, 0)$ in y-Richtung. In der folgenden Implementierung ist `dist1` der Manhattan-Abstand von `this` von

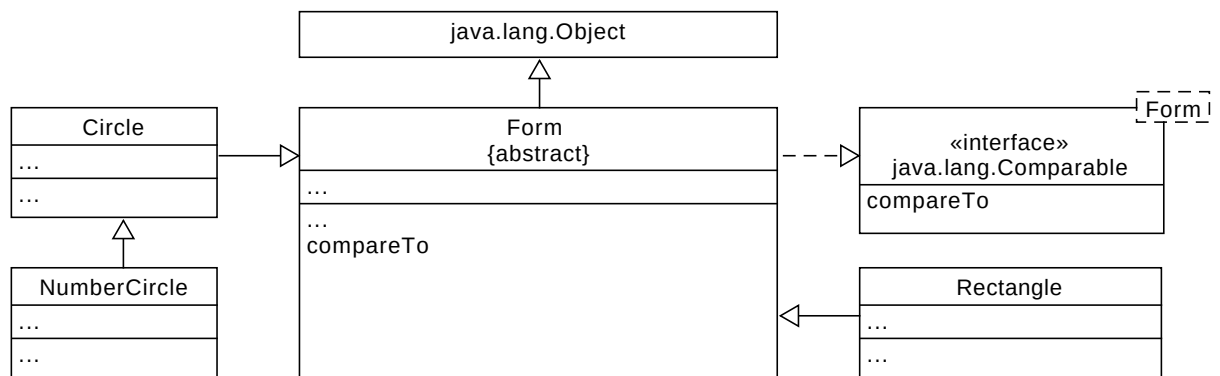


Abbildung 10: Klasse Form implementiert die API-Schnittstelle Comparable<Form>.

(0, 0) und `dist2` der Manhattan-Abstand von `o` von (0, 0). Je nachdem, ob `dist1` oder `dist2` größer ist, wird 1 oder -1 zurückgegeben. Sind `dist1` und `dist2` gleich groß, wird 0 zurückgegeben.

```

public int compareTo(Form o) {
    int dist1 = this.getPosition().getX() + this.getPosition().getY()
    ;
    int dist2 = o.getPosition().getX() + o.getPosition().getY();
    if (dist1 - dist2 < 0)
        return -1;
    if (dist1 - dist2 > 0)
        return 1;
    return 0;
}

```

Das ist übrigens nicht die einzige mögliche korrekte Implementierung. Auch die folgende Implementierung würde zum Beispiel den Vorgaben aus der API entsprechen:

```

public int compareTo(Form o) {
    int dist1 = this.getPosition().getX() + this.getPosition().getY()
    ;
    int dist2 = o.getPosition().getX() + o.getPosition().getY();
    return dist1 - dist2;
}

```

Jetzt können wir als Anwendung wie folgt in einem Array gespeicherte Formen sortiert ausgeben:

```

Form[] allForms;
// Array initialisieren / Formen hinzufuegen: siehe Unterkapitel 4.5
Arrays.sort(allForms);
System.out.println(Arrays.toString(allForms));

```

Die API-Klasse `java.util.Arrays` ist eine Utility-Klasse, die etliche nützliche statische Methoden zur Verwaltung von Arrays zur Verfügung stellt. Zum Beispiel lassen sich Arrays mit der Methode `sort` sortieren. Für die Sortierung wird intern die Methode `compareTo` verwendet. Voraussetzung dafür ist also, dass alle Elemente des Arrays die Schnittstelle `Comparable` implementieren. Außerdem gibt es die Methode `toString`, um den Inhalt aller Elemente des Arrays in einen String umzuwandeln.

4.4.1 Deklaration und Definition von Schnittstellen

Beispiele für selbst entworfene Schnittstellen lernen wir erst später in der Vorlesung kennen. Trotzdem definieren wir hier Schnittstellen allgemein. Dadurch sind Sie ab jetzt in der Lage, diese in der API korrekt zu interpretieren und anzuwenden.

Definition 4.42: Deklaration von Schnittstellen

Schnittstellen in Java sind Datentypen und werden wie folgt deklariert:

```
[<Sichtbarkeit>] interface <Schnittstelle>
    [ extends <Oberschnittstelle> ]
```

Dabei sind die einzelnen Bestandteile wie folgt definiert:

- `<Sichtbarkeit>`: Die Sichtbarkeit der Schnittstelle.
- `<Schnittstelle>`: Der Name der Schnittstelle.
- `extends <Oberschnittstelle>`: Festlegung einer sogenannten **Oberschnittstelle** – von einer solchen werden alle Methoden geerbt.

Definition 4.43: Sichtbarkeit von Schnittstellen

Folgende Sichtbarkeiten sind möglich:^a

- `public`: Der Zugriff auf die Schnittstelle ist in allen anderen Klassen möglich.
- `package private`: Der Zugriff auf die Schnittstelle ist nur in Klassen möglich, die sich in dem gleichen Paket wie diese Klasse befinden. Das ist der Standardfall, wenn man keine Sichtbarkeit angibt.

^aZwar ist es auch möglich, Schnittstellen als `private` oder `protected` zu deklarieren, allerdings ist dies nur erlaubt, wenn die Schnittstelle im Rumpf einer Klasse definiert wurde (das ist dann eine sogenannte innere Schnittstelle, solche betrachten wir aber nicht weiter).

Eine Schnittstelle wird (wie eine Klasse) definiert durch ihre Deklaration und ihren **Rumpf**. Dieser enthält (wie bei Klassen) die Implementierung und wird nach der Deklaration in geschweiften Klammern angegeben.

Definition 4.44: Rumpf von Schnittstellen

Der Rumpf einer Schnittstelle kann enthalten:

Öffentliche statische Methoden (mit Implementierung, sind implizit `public`, seit Java 8):

```
static <Typ> <Methode>(<Parameterliste>) {<Rumpf>}
```

Öffentliche Default-Methoden (mit Implementierung, sind implizit `public`, seit Java 8):

```
default <Typ> <Methode>(<Parameterliste>) {<Rumpf>}
```

Öffentliche abstrakte Objekt-Methoden (ohne Implementierung, sind implizit `public abstract`):

```
<Typ> <Methode>(<Parameterliste>);
```

Öffentliche statische Konstanten (sind implizit `public static final`):

```
<Typ> <Konstante> = <Wert>;
```

In UML sieht eine schematische Darstellung von Schnittstellen und implementierenden Klassen wie in Abbildung 11 aus.

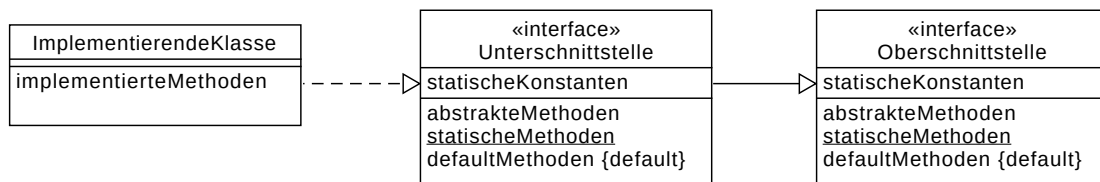


Abbildung 11: Schematische Darstellung von Schnittstellen und implementierenden Klassen in UML.

Statische Methoden werden wie bei Klassen unterstrichen. Default-Methoden werden mit dem Schlüsselwort `default` gekennzeichnet.

Schnittstellen enthalten im Gegensatz zu Klassen **keine Objektattribute, keine privaten Attribute und Methoden** und **keine Konstruktoren**. Eine Schnittstelle soll lediglich dafür sorgen, dass alle in ihr deklarierten Methoden und Konstanten in den Klassen zur Verfügung stehen, **die sie implementieren**. Eine Schnittstelle kann dabei von mehreren Klassen implementiert werden.

Definition 4.45: Klassen implementieren Schnittstellen

Eine Klasse `K` implementiert wie folgt Schnittstellen `S1`, `S2`, ...:

```
K implements S1, S2, ...
```

In diesem Fall gilt:

- `K` muss **alle abstrakten Methoden** der Schnittstellen `S1`, `S2`, ... konkret implementieren. Durch die abstrakten Methoden werden also **gemeinsame Methodenköpfe** für alle implementierenden Klassen zur Verfügung gestellt.
- Die **Default-Methoden** von `S1`, `S2`, ... stehen als Objektmethoden von `K` zur Verfügung. Sie enthalten einen Implementierungsvorschlag, also eine Möglichkeit, wie die Methode sinnvoll implementiert sein könnte. `K` kann die Default-Implementierung übernehmen oder sie überschreiben.
- Die statischen Methoden von `S1`, `S2`, ... können wie Klassenmethoden von `K` verwendet werden.
- Die statischen Konstanten von `S1`, `S2`, ... können wie Klassenkonstanten von `K` verwendet werden.

Neben abstrakten Methoden enthält die Schnittstelle `CharSequence` auch statische Methoden und Default-Methoden:

- `default boolean isEmpty()`: Default-Methode für den Test, ob eine Zeichenfolge leer ist – steht also als Objektmethode in `String` und weiteren implementierenden Klassen zur Verfügung.
- `static int compare(CharSequence cs1, CharSequence cs2)`: Statische Methode zum Vergleich von zwei Zeichenfolgen – steht also als Klassenmethode in `String` und weiteren implementierenden Klassen zur Verfügung.

Übung: Überlegen Sie, wie die Implementierung dieser beiden Methoden auf Ebene der Schnittstelle aussehen könnte. Hierbei dürfen nur (andere) Methoden der Schnittstelle verwendet werden! Lesen Sie dann deren Standard-Implementierung in der API nach.

4.4.2 Schnittstellen als Datentypen

Einen sehr wichtigen Aspekt von Schnittstellen haben Sie bisher vielleicht überlesen:

Schnittstellen sind Datentypen!

Das bedeutet zum Beispiel, dass wir Variablen vom Typ einer Schnittstelle deklarieren können:

```
CharSequence seq;
```

Für `seq` stehen dann genau die Methoden der Schnittstelle `CharSequence` zur Verfügung. Das kann Sinn machen, weil `CharSequence` der Rückgabotyp von Methoden ist, wie zum Beispiel der folgenden Methode der Schnittstelle `CharSequence`:

```
CharSequence subSequence(int start, int end)
```

Diese Methode liefert die Teilfolge aus Zeichen von Index `start` bis zum Index `end-1` und kann wie folgt benutzt werden:

```
String s = new String("Hello");  
CharSequence seq = s.subSequence(0, 2) // liefert "He"
```

Nach dem Substitutionsprinzip kann `seq` aber auch Inhalt referenzieren, dessen Typ eine Klasse ist, die `CharSequence` implementiert (wie zum Beispiel `String`):

```
CharSequence seq = new String("Hello");
```

Dasselbe trifft auch auf Eingabeparameter zu. Die Methode

```
boolean contains(CharSequence s)
```

der Klasse `String` überprüft, ob ein String `s` enthält. Übergeben werden können Objekte aller Klassen, die die Schnittstelle `CharSequence` implementieren (wie zum Beispiel auch `StringBuilder`-Objekte). Das funktioniert, weil in der Implementierung von `contains` nur `CharSequence`-Methoden verwendet werden.

Definition 4.46: Schnittstellen als Datentypen benutzen

Implementiert eine Klasse `K` eine Schnittstelle `S`, so ist jedes Objekt vom Typ `K` auch vom Typ `S`.

Nach dem **Substitutionsprinzip** kann eine `S`-Variable Inhalt vom Typ `K` referenzieren. Für diese Variable stehen dann aber nur die `S`-Methoden zur Verfügung.

Die Typzugehörigkeit zu `S` kann mit `instanceof` überprüft werden.

Eine Schnittstelle `S` definiert damit eine **eingeschränkte (spezifische) Sicht auf die Benutzbarkeit von Objekten**: Ist eine Variable `s` vom Typ `S`, so stehen für jedes für `s` eingesetzte Objekt nur die `S`-Methoden zur Verfügung, auch wenn der Typ des Objekts noch wesentlich mehr Methoden hat. Von diesen anderen Methoden wird abstrahiert.

Indem man die Typen von Methodenparametern möglichst eingeschränkt wählt, lassen sich **verschiedene Bereiche einer Implementierung entkoppeln**: Benötigt man in einer Methode für ein übergebenes Objekt nur `S`-Methoden, so verwendet man für den Eingabeparameter den Typ `S`. Für den Eingabeparameter lassen sich dann **viele unterschiedliche Objekte einsetzen**, nämlich von allen (auch neu definierten) Klassen, die die Schnittstelle `S` implementieren. **Die Methode wird dadurch allgemeiner benutzbar.**

Zudem ist die Implementierung der Methode nur abhängig von `S`, aber nicht von anderen Eigenschaften der eingesetzten Objekte und nicht davon, *wie* die Objekte die `S`-Methoden implementieren. Dadurch sinkt die Gefahr von Nebeneffekten zwischen Eigenschaften der eingesetzten Objekte und der Funktionsweise der Methode (zum Beispiel bei Änderungen am Code der Objekte).

Mehr Beispiele zu Schnittstellen und deren Benutzung finden Sie in *Screencast 05: Die Oberklasse ist tot, lang lebe die Schnittstelle!*.

4.5 Arrays

Ein **Array** ist eine Variable mit endlich vielen Elementen vom gleichen Datentyp. Jedes Element ist dann eine Variable dieses Datentyps. In einem Array können wir also mehrere Werte eines einheitlichen Datentyps speichern. Als Datentyp kommen beliebige primitive Datentypen und Referenz-Datentypen in Frage. Möchten wir zum Beispiel eine Liste der Zahlen 2, 4, 6, 8 (in dieser Reihenfolge) speichern, können wir mit der Anweisung

```
int[] even;
```

ein Array mit dem Namen `even` deklarieren, das diese Werte speichern soll. Mit Hilfe der Deklaration wird Speicher für eine Referenz reserviert, aber noch nicht der Speicherbereich für die Daten, die in dem Array stehen sollen. Das liegt daran, dass wir noch gar nicht gesagt haben, wie viele Elemente wir in unserem Array stehen haben wollen. Das können wir nachholen, indem wir unser Array mit der Anweisung

```
even = new int[4];
```

initialisieren. Auf der rechten Seite steht ein Konstruktoraufruf für ein Array vom Typ `int`. In die eckigen Klammern schreiben wir die Anzahl der gewünschten Elemente. Hier wird also genügend Speicherplatz für 4 Datenwerte vom Typ `int` reserviert. Wie bei Klassen liefert der Konstruktor die Speicheradresse des reservierten Speicherbereichs. Diese Adresse wird durch die Wertzuweisung als Referenz in `even` gespeichert.

Arrays sind also wie Klassen Referenzdatentypen!

Bei der Initialisierung werden die Elemente des Arrays außerdem auf den (zum Datentyp gehörenden) Standardwert gesetzt. Das ist 0 beim Datentyp `int`. Wir können uns unser Array im Speicher dann wie in Abbildung 12 vorstellen.

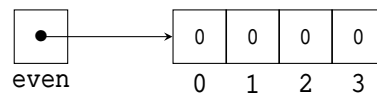


Abbildung 12: Ein Array im Arbeitsspeicher nach Konstruktoraufruf.

Hierbei stellen die Rechtecke Speicherzellen dar, in die die gespeicherten Wert eingetragen sind. Unter den Speicherzellen steht der Index, mit dem wir auf das jeweilige Element zugreifen können. Die Referenz (Speicheradresse) ist als Pfeil dargestellt. Wir können nun mit einer `for`-Schleife den reservierten Speicherbereich durchlaufen und die Datenwerte setzen, die wir uns an den entsprechenden Stellen wünschen:

```
for (int i = 1; i <= 4; i++) {
    even[i-1] = 2*i;
}
```

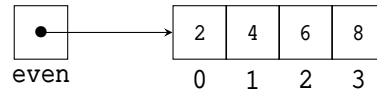


Abbildung 13: Ein Array im Arbeitsspeicher nach Wertzuweisung.

Mit dem Ausdruck

```
even[i-1]
```

greifen wir hierbei auf das i -te Element des Arrays zu. Das heißt, dass der erste Eintrag am Index 0 steht, der zweite Eintrag am Index 1, und so weiter. Hierbei entspricht `even[i-1]` einer `int`-Variable, der wir Werte zuweisen oder die wir in Rechenausdrücken verwenden können.

Alternativ kann man Arrays auch initialisieren, indem man gleich explizit die Werte angibt, die in dem Array stehen sollen (die Reihenfolge ist dabei wichtig). Allgemein kann man schreiben:

```
<Array> = {val1, val2, ... valN};
```

wobei `val1`, ..., `valN` die Werte sind, die in dem Array stehen sollen. In unserem Beispiel können wir also anstelle des Konstruktoraufrufs und der `for`-Schleife auch schreiben:

```
even = {2, 4, 6, 8};
```

Die interne Repräsentation unseres Arrays können wir uns im Anschluss wie in Abbildung 13 vorstellen.

Anstelle eines primitiven Datentyps wie `int` können wir auch Referenz-Datentypen als Typ für die Elemente eines Arrays verwenden. Damit können wir zum Beispiel Arrays von `String`-Objekten bilden. Genau so ein `String`-Array ist der Eingabeparameter `args` von `main`. Oder wir können Arrays von `Form`-Objekten bilden, wie schon im letzten Unterkapitel angedeutet. Das sieht dann wie folgt aus:

```
Form[] allForms;
allForms = new Form[2];
allForms[0] = new Circle(50, new Point(10, 20));
allForms[1] = new Rectangle(10, 10, new Point(20, 10));
```

Hier wird ein Array angelegt, der zwei `Form`-Objekte aufnehmen kann. Als erstes Element wird ein `Circle`-Objekt und als zweites Element ein `Rectangle`-Objekt gespeichert.

Es ist aber auch möglich, dass die Elemente eines Arrays wieder Arrays sind. In diesem Fall spricht man von **mehrdimensionalen Arrays**, sonst von **eindimensionalen Arrays**.

Erstmal sehen wir uns nun eindimensionale Arrays etwas formaler an.

Definition 4.47: Deklaration von 1-dimensionalen Arrays

Ein **1-dimensionales Array** `array` vom Typ `T` wird wie folgt deklariert:

```
T[] array;
```

Hier gilt

- `T` ist der **Datentyp der Elemente** des Arrays.
- `array` ist der **Name** des Arrays.
- `T[]` ist der **Datentyp von `array`**. Dieser wird auch für array-wertige Rückgabetypen von Methoden verwendet.
- Es wird Speicher für die Referenz `array` reserviert und diese mit `null` initialisiert.
- Die Anzahl der Arrayelemente wird erst bei der Initialisierung festgelegt.
- Die obige Deklaration wird auch für array-wertige Eingabeparameter von Methoden verwendet.

Definition 4.48: Initialisierung von 1-dimensionalen Arrays

Ein 1-dimensionales Array `array` vom Typ `T` der Länge `n` wird wie folgt über einen **Konstruktoraufruf** initialisiert:

```
array = new T[n];
```

Dies sorgt dafür, dass Speicherplatz für `n` viele Elemente des Arrays reserviert wird und `array` eine Referenz auf diesen Speicherplatz ist. Der Wert `n` darf zur Laufzeit festgelegt werden, kann aber danach nicht mehr verändert werden.

Definition 4.49: Zugriff auf 1-dimensionalen Arrays

Auf das `i`-te Element eines Arrays `array` vom Typ `T` wird mit

```
array[i-1]
```

zugegriffen. `array[i-1]` ist dabei ein Variable vom Datentyp `T`. Das erste Element steht an Index 0. Hat ein Array `n` Elemente, so ist `n-1` der Index des letzten Elements des Arrays.

Die Länge eines Arrays kann über das Attribut `length` abgefragt werden:

```
array.length
```

Ein solches Attribut wird für jedes Array automatisch von Java erstellt, wir müssen es also nicht zusätzlich setzen – es genügt, die Arraylänge bei der Initialisierung anzugeben.

Mit den Arrays, die wir hier kennengelernt haben, ist es nicht möglich, die Länge während der Laufzeit zu verändern. In der Java-API gibt es aber verschiedene vordefinierte Bibliotheksklassen

(sogenannte Sammlungen) zur Aufnahme einer variablen Anzahl von Elementen, zum Beispiel [ArrayList](#), [TreeSet](#), [HashMap](#), und mehr. Sie unterscheiden sich in ihrer internen Implementierung und haben daher verschiedene Vor- und Nachteile. Wir lernen sie nach und nach genauer kennen.

Für einen kleinen Vergleich zwischen „normalen“ Arrays und der Bibliotheksklasse [ArrayList](#) sei an dieser Stelle noch einmal auf *Screencast 02: Mein bester Freund, die API* verwiesen, speziell die zweite Hälfte dieses Screencasts.

4.5.1 Mehrdimensionale Arrays

Ein mehrdimensionales Array hat Elemente, die wieder Arrays sind. In den meisten Anwendungsfällen kommt man mit 2-dimensionalen Arrays aus. Mit der Anweisung

```
int[][] matrix = new int[2][4];
```

deklarieren wir ein 2-dimensionales Array [matrix](#) vom Typ [int](#). Dieses wird durch Konstruktorauf-ruf initialisiert mit 2 Elementen (erste eckige Klammer). Jedes Element ist wieder ein Array und hat 4 Elemente vom Typ [int](#) (zweite eckige Klammer). Im Speicher können wir uns die Situation wie in Abbildung 14 vorstellen.

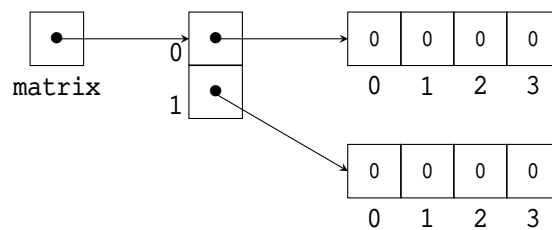


Abbildung 14: Ein 2-dimensionales Array im Arbeitsspeicher nach Konstruktoraufruf.

Durch die Deklaration wird also Speicherplatz für die Referenz [matrix](#) reserviert, die wiederum eine Liste von Arrays referenziert. Für die Initialisierung müssen wir einerseits festlegen, wie viele Elemente [matrix](#) hat, und wie viele Elemente wiederum jedes Element hat (denn jedes Element ist ja wieder ein Array).

Definition 4.50: Deklaration und Initialisierung von 2-dimensionalen Arrays

Ein 2-dimensionales Array `matrix` vom Typ `T` wird wie folgt deklariert:

```
T[][] matrix;
```

Ein Array `matrix` der Länge `n` und Breite `m` wird wie folgt initialisiert:

```
matrix = new T[n][m];
```

Hierbei gilt:

- `T[][]` ist der **Datentyp** von `matrix`. Dieser wird auch für array-wertige Rückgabetypen von Methoden verwendet.
- Die obige Deklaration wird auch für array-wertige Eingabeparameter von Methoden verwendet.
- `n` ist die Anzahl der Elemente von `matrix`. Diese kann über `matrix.length` abgefragt werden.
- `matrix[i-1]` ist ein 1-dimensionales Array vom Typ `T` und entspricht dem `i`-ten Element von `matrix`.
- `m` ist die Anzahl der Elemente von `matrix[i-1]` (für jedes `i`). Diese kann über `matrix[i-1].length` abgefragt werden.
- `matrix[i-1][j-1]` ist eine Variable vom Typ `T` und entspricht dem `j`-ten Element von `matrix[i-1]`.

Wir können uns ein 2-dimensionales Array wie eine Matrix vorstellen. Jedes Element des Arrays entspricht einer Zeile der Matrix. Die Anzahl der Elemente ist also die Anzahl der Zeilen der Matrix. Die Anzahl der Elemente jeder Zeile ist die Anzahl der Spalten der Matrix.

Nach der Initialisierung können wir die einzelnen Elemente der Matrix mit Werten belegen. Das kann man einerseits über eine Doppelschleife erledigen:

```
for (int i = 1; i <= 2; i++) {  
    for (int j = 1; j <= 4; j++) {  
        matrix[i-1][j-1] = i * j;  
    }  
}
```

Und es gibt wieder folgende alternative Möglichkeit (für primitive Datentypen):

```
<Matrix> = {{v11, v12, ..., v1N}, {v21, v22, ..., v2N}, ... };
```

Anstelle von Werten eines primitiven Datentyps geben wir hier also jeweils Arrays in jedes Array-Element an. Probieren wir dies an dem Beispiel unserer `int`-Matrix aus:

```
matrix = {{1, 2, 3, 4}, {2, 4, 6, 8}};
```

Jetzt können wir uns die Situation im Speicher wie in Abbildung 15 vorstellen.

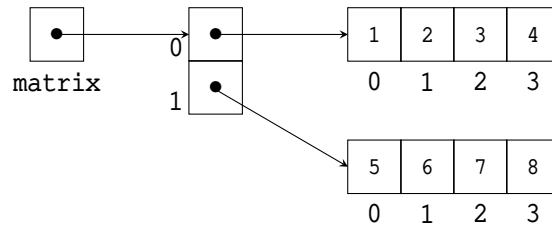


Abbildung 15: Ein 2-dimensionales Array im Arbeitsspeicher nach Wertzuweisung.

4.5.2 Die Klasse Arrays

Die Klasse `Arrays` stellt einige praktische statische Hilfsmethoden für Arrays zur Verfügung. Es folgt eine unvollständige Übersicht:

- `toString`: Liefert alle Elemente eines Arrays als Strings.
- `deepToString`: Liefert alle Elemente eines Arrays als Strings für mehrdimensionale Arrays.
- `sort`: Sortiert ein Array. Dafür müssen die Elemente entweder die Schnittstelle `Comparable` implementieren oder es muss ein `Comparator`-Objekt übergeben werden (eigene API-Recherche).
- `equals`: Untersucht zwei Arrays auf Gleichheit, also ob sie dieselbe Länge haben und sie identische Elemente in derselben Reihenfolge enthalten (Sonderfall: Zwei Array-Referenzen sind ebenfalls gleich, wenn sie beide den Wert `null` haben).
- `deepEquals`: Wie `equals`, aber für mehrdimensionale Arrays.
- `fill`: Belegt alle Elemente mit gleichem Wert.
- `setAll`: Belegt alle Elemente mit Werten (mit Übergabe eines Lambda-Ausdrucks¹ zur Berechnung der Werte.)
- `copyOf`, `copyOfRange`: kopiert ein Array oder Teile davon.
- `binarySearch`: Führt eine binäre Suche durch.
- `asList`: Wandelt ein Array in ein `List`-Objekt um.
- Einige Methoden für schnelle parallele Berechnungen.

Diese Übersicht enthält nur die Namen der Methoden und keine vollständigen Methodenköpfe, da die Methoden in der Regel überladen sind. Für eine detaillierte Übersicht und vollständige Methodenköpfe empfiehlt sich ein Blick in die Java-API.

4.5.3 Vergleich von Java und C

Im Vergleich zu Arrays in C gibt es folgende wichtige Unterschiede in Java:

- Die Länge eines Arrays kann über das Attribut `length` abgefragt werden.
- Anders als in C werden die Werte in den reservierten Speicherzellen automatisch mit dem zum Datentyp passenden Nullwert initialisiert.
- In Java ist keine explizite Zeigerarithmetik (Adressverschiebung) möglich, da Java-Programme plattformunabhängig sein sollen.

¹Eine kompakte Schreibweise für eine Methode, die irgendwo eingesetzt werden soll. Details folgen im Kapitel 5.